



Mobiliser Framework

Architecture and Design

SAP Mobile Platform - Mobiliser 5.5

Copyright © 2015 SAP AG. All rights reserved.

SAP, R/3, SAP NetWeaver, Duet, PartnerEdge, ByDesign, SAP BusinessObjects Explorer, StreamWork, SAP HANA, and other SAP products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of SAP AG in Germany and other countries.

Business Objects and the Business Objects logo, BusinessObjects, Crystal Reports, Crystal Decisions, Web Intelligence, Xcelsius, and other Business Objects products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of Business Objects Software Ltd. Business Objects is an SAP company.

Sybase and Adaptive Server, iAnywhere, Sybase 365, SQL Anywhere, and other Sybase products and services mentioned herein as well as their respective logos are trademarks or registered trademarks of Sybase Inc. Sybase is an SAP company.

Crossgate, m@gic EDDY, B2B 360, and B2B 360 Services are registered trademarks of Crossgate AG in Germany and other countries. Crossgate is an SAP company.

All other product and service names mentioned are the trademarks of their respective companies. Data contained in this document serves informational purposes only. National product specifications may vary.

These materials are subject to change without notice. These materials are provided by SAP AG and its affiliated companies ("SAP Group") for informational purposes only, without representation or warranty of any kind, and SAP Group shall not be liable for errors or omissions with respect to the materials. The only warranties for SAP Group products and services are those that are set forth in the express warranty statements accompanying such products and services, if any. Nothing herein should be construed as constituting an additional warranty.



Contents

1	Introduction	7
2	Mobiliser Gateway	8
2.1	Overview	8
2.2	Protocols	10
2.2.1	Technologies	10
2.2.2	OSGi Dynamics	10
2.2.3	REST	11
2.2.4	SOAP	12
2.2.5	JMS	13
2.2.6	TCP	13
2.2.7	OData	13
2.2.8	Clients	13
2.3	Security	14
2.3.1	Authentication	14
2.3.2	Authorisation	17
2.4	Request Repetition	20
3	Messaging Framework	22
3.1	Message Gateway	22
3.1.1	Messages	22
3.2	Templating Engine	23
3.3	Channel Manager	24
3.3.1	Overview	24
3.3.2	API	25
4	Preferences	30
4.1	Structure	30
4.2	API	30
4.3	Access	31

4.3.1	IBackingStore	31
4.3.2	Interceptors	31
4.4	Refresh and Events	32
5	Persistence	35
5.1	Data Model Beans	35
5.2	Data Access Object API	35
5.3	Data Access Object Implementation	36
5.3.1	Persistence Service Provider	36
5.3.2	DAO Factory	37
6	Reports	38
6.1	Introduction	38
6.2	Mobiliser Reporting Framework	39
6.3	On-line Reports (Ad-hoc Reports)	40
6.4	Asynchronous Reports	40
6.4.1	Report Services	41
6.4.2	Report Job	41
6.5	Report Store	41
7	Events	43
7.1	System Overview	43
7.2	Event Generation	46
7.2.1	Disable Event Generation	46
7.3	Event Processing	47
7.3.1	Event Generation Processing	47
7.4	Event Dispatch Processing	48
7.4.1	Handler/Event Process Lock	49
7.4.2	Processing of Scheduled Events	49
7.5	Event Handling	50
7.5.1	Event Handler Registration	50
7.5.2	Event Handler Polling	50
7.6	Event Catchup	51
7.7	Task Configuration	52
7.7.1	Task Handler Registration	52
7.7.2	Task Configuration	53
7.7.3	Task Processing	54

List of Figures

2.1	Gateway Architecture	9
2.2	HTTP Request Sequence Diagram	14
2.3	HTTP Request Security Sequence Diagram	18
2.4	Example RunAsManager Sequence Diagram	20
2.5	Request Interruption	21
3.1	Messaging Framework Architecture	23
4.1	Example Preferences Trees	31
4.2	Sequence Diagram Preferences Refresh + Listener	33
4.3	Sequence Diagram Get Preferences Value	34
6.1	The Crystal Reports Embedded Reporting Architecture	39
6.2	Mobiliser Reporting Architecture	40
6.3	Directory tree for the report store, templates and archives	42
7.1	Overall System Overview	45
7.2	Generation Class Model Overview	46
7.3	Event Generation Processing	47
7.4	Event Dispatch Processing #1	48
7.5	Event Dispatch Processing #2	49
7.6	Event Handler Registration	50
7.7	Event Handler Pooling	51
7.8	Event Handler Catchup	52
7.9	Task Handler Registration	53

List of Tables

5.1	Base DAO interfaces	36
6.1	Common Report Parameters	42

Chapter 1

Introduction

The Mobiliser Framework Architecture and Design document covers the Mobiliser Framework. It describes the overall architecture and the core components provided by the Mobiliser platform

- Mobiliser Service Gateway
- Messaging Framework
- Preferences
- Persistence
- Reports and
- Events.

It does not cover the Money Mobiliser specifics and also does not provide information on how to implement and add new customized components.

Please see “Money Mobiliser Architecture and Businesslogic” for details on the additional components provided by Money Mobiliser and “Mobiliser Framework Development Guide” when extending the Mobiliser Framework.

This document’s target audience is business analysts, project managers and developers who need details on the Mobiliser Framework. This document is technical in nature and does not describe the business cases behind Mobiliser installations. A good general technical background is helpful to understand the more detailed aspects described in this document. However, each module is introduced conceptually as well before going into the details, which does not require any specific knowledge.

Chapter 2

Mobiliser Gateway

The Mobiliser gateway is a loose term we use to describe the infrastructure in place to expose services to external systems. While that is indeed vague, the gateway itself is just a thin facade to wrap different protocols under a common interface. It also provides common interfaces for defining security for services, which privileges are need by callers if any, and to do this all dynamically at runtime in a protocol independent way.

2.1 Overview

The overall design of the gateway is to define services through an XSD schema and write endpoints accepting these messages which can be published through any protocol.

To achieve these goals, we have set up a few constraints on what an exposed service should look like. Messages are always defined as pairs of requests and responses. These are mapped in Java to a single Java method which takes the request as a method parameter and returns the response. The protocol specific implementations will be discussed later.

This is nothing new and is just the classic “contract first” approach to web services. This allows us to define the contract for our services, the actual constraints on incoming data in XSD, which is particularly suited for this task, and then independently choose the implementation and protocol we will use for it.

In Mobiliser nomenclature we will speak of some of the following terms:

Context This simply means a collections of endpoints grouped together in some way. In a typical installation, endpoints will be grouped to have common operations together.

Mapping Mappings are at the protocol level and map an incoming request onto a particular endpoint object.

Adapter Adapters are also at the protocol level and adapt incoming requests in some way to allow calling the endpoint methods.

Contract This is simply the contract for the endpoint as expressed in XSD.

Endpoint This is the actual piece “service” which performs some operation. What appears to the outside as a single service may actually be multiple endpoints grouped into one.

“Standard” Mobiliser services will adhere to this convention. The gateway itself will however accept any mapping or adapter so implementing custom services for a client who cannot speak to the standard services is also trivial. An example of this is used in Channel Manager to allow exposing custom services over HTTP, but not using the “Standard” Mobiliser XSD requests and responses.

2.2 Protocols

A default Mobiliser installation supports three HTTP styles out of the box. Traditional SOAP services over HTTP, “RESTful” services where the payload is sent to the server as an HTTP post¹, and an OData interface. The gateway natively supports JSON, XML, and special Java serialized payloads for these POST requests. We say “RESTful” in quotes since the requests are always POSTs and do not support other HTTP verbs or URI parameters, so it is not true REST.

SOAP messages may also be submitted via JMS. In that case, the actual SOAP-XML is submitted to a named JMS queue as a `TextMessage` or `BytesMessage` and then follows the same path as if submitted via HTTP (excluding any HTTP filters). The context name is used as the queue name.

The SOAP support allows building dynamic WSDLs. As mentioned before, groups of endpoints are grouped together into contexts and each context will collect the XSDs of the endpoints registered with it. These are then used to generate a dynamic WSDL on the fly.

The plain XML messages accepted by the restful endpoints can also be consumed via pure TCP. The TCP integration does not specify contexts, therefore all services are accessible simply through the TCP port. Since some payload QNames are not unique across all contexts, certain contexts may be filtered – in Vanilla, for example, the Smartphone services are ignored.

The OData interface exposes standard CRUD operations already implemented by the standard services in an OData compliant way.

2.2.1 Technologies

The gateway stack simply builds upon Spring technologies. The SOAP integration uses Spring-WS², while the HTTP POST services use Spring-MVC³. The TCP protocol is built on top of Sprint-Integration⁴. Since these technologies form the basis of the gateway, we can add support for new protocols simply by implementing new adapters and mappings for those protocols. We are using JAX-B for XML marshalling, Jackson⁵ as our JSON mapper and simple `ObjectOutputStreams` and `ObjectInputStreams` for the java serialized payloads. We chose Jackson for its support of JAX-B annotation introspection, which allows us to reuse our contract beans generated by XJC without any additional work. The OData stack is built on top of odata4j⁶.

2.2.2 OSGi Dynamics

Referring to the gateway architecture diagram, these pieces are all standard Spring components. Normal Spring applications would configure them statically and these would work in the same way. We have added extensions to each - Spring Security, Spring-WS, and Spring-MVC - which allows us to register them at runtime through OSGi exports.

For HTTP access, whether SOAP, the simple HTTP Post requests or OData, we are using Spring-MVC's standard `DispatcherServlet`. In a traditional war packaged web application, we would either configure our mappings and adapters explicitly in a Spring XML file or configure package scanning in the XML file. This would require us to know at startup which services we wish to publish and also have them available at startup. Since the gateway must allow new services to be registered or disabled at runtime, we extended the servlet to allow them to be added or removed dynamically.

We may use this type of integration and have each bundle, which wants to publish a service, provide an adapter and a mapping for it. The disadvantage here is that every service bundle would have to know that we are using Spring-WS, Spring-MVC and that we want to allow SOAP, JSON, XML, and serialized Java access to the service. We also lose support for JMS access unless it is also configured there. To alleviate this problem,

¹Yes, SOAP is basically a special XML POST request, but wrapped in the SOAP envelope.

²Spring-WS <http://static.springsource.org/spring-ws/sites/2.0/>

³Spring-MVC <http://static.springsource.org/spring/docs/3.0.x/spring-framework-reference/html/mvc.html>

⁴Spring-Integration <http://www.springsource.org/spring-integration/>

⁵Jackson <http://jackson.codehaus.org/>

⁶odata4j <https://code.google.com/p/odata4j/>

endpoint bundles export a special object called an `IEndpointInformation` object, which describes the endpoint itself.

```
1 public interface IEndpointInformation {
2
3     /**
4      * Return the endpoint instance. This is the actual object which can process
5      * a request and return a response.
6      *
7      * @return the endpoint instance
8      */
9     Object getEndpoint();
10
11     /**
12      * Return the paths to the xsd schemas needed by this endpoint. These should
13      * be the full paths to the schemas on the classpath.
14      *
15      * @return the list of schemas
16      */
17     String[] getSchemaPaths();
18
19     /**
20      * Return a list of regex values to specify which elements from the given
21      * schemas are processed by this endpoint. You can split up the messages in
22      * the schema between multiple endpoints. Each endpoint should then only
23      * have the elements it processes in this list.
24      *
25      * @return the list of message element regexes
26      */
27     String[] getAllowedMessageElements();
28
29     /**
30      * Return the paths needed to build the JAX-B context for this endpoint.
31      * This is a list of package names.
32      *
33      * @return the list of package names.
34      */
35     String[] getContextPaths();
36 }
```

Listing 2.1: Endpoint Information API

This allows listening for these objects and then registering the required objects to make the rest of the integration work.

2.2.3 REST

The “RESTful” integration is the simplest since requests are mapped based on URLs. It is used to handle XML, JSON, and serialized Java objects. The only supported HTTP verb is POST, and the payload must be posted to URLs under `/mobiliser/rest`. To determine the operation to call, a POST request would be sent to the name of the context plus some unique operation name, e.g. `/mobiliser/rest/transaction/authorise`. This path is configurable in the gateway and this convention is currently used by the Mobiliser framework. Additionally, Mobiliser generates a Javascript client dynamically that can be used to invoke Mobiliser services via JSON POST calls at, e.g., `/mobiliser/rest/transaction?js`. A WADL for the service will also be generated dynamically, e.g. `/mobiliser/rest/transaction?wadl`.

The gateway has one bundle in the container for the context “transaction” which registers an `OSGiHandlerMapping` and an `OSGiHandlerAdapter` which will then be picked up by the extended servlet. These objects listen for any `EndpointInformation` objects registered for the context “transaction” and for each it adds delegate mappings and adapters.

To create the delegate mappings, the exported endpoint from the `EndpointInformation` object is introspected

and all methods that match the requirement of having a single JAXB object argument and returning a JAXB object are used. The mapping is the name of the method and it maps onto the endpoint object from `EndpointInformation`. In Spring-MVC that object is called a handler, since it handles the request.

At the same time, handler adapters are registered which take the request, create a JAX-B, call the endpoint method with that object, and write the response JAX-B bean back into the response output stream. This is why the parameters needed to create a JAX-B context and a schema object are included in the `EndpointInformation`, since the incoming data is validated. Three adapters are registered in the standard deployment - one which reads and writes XML using a JAX-B marshaller and un-marshaller, one which reads and writes JSON data using Jackson, and one which reads and writes serialized Java objects using object input and output streams. The clients specifies this in the Content-Type and Accept headers sent with the request (application/xml, application/json, application/x-java-serialized-object).

2.2.4 SOAP

SOAP requests are slightly more complex. All requests are sent to a single URL determined by the context name. So, using the same example as in the REST section, to `/mobiliser/transaction`. This path is also configurable, but that is the convention currently used.

Since SOAP requests are processed by Spring-WS, we bridge the Spring-MVC and Spring-WS APIs. Another bundle which includes the SOAP support for the gateway registers a special mapping object `SybaseUrlHandlerMapping`. This object maps solely based on URLs and listens for `WebServiceMessageReceiver` objects to be registered for a URL and dynamically adds them to the mapping.

The bundle also defines the standard adapter `WebServiceMessageReceiverHandlerAdapter` which allows passing the incoming HTTP request to the `WebServiceMessageReceiver`.

The “transaction” context bundle we registered above will also register one of these `WebServiceMessageReceivers`. This is the actual handler seen by the servlet as mentioned previously.

The context bundle also registers two special Spring-WS objects, one for Spring-WS mapping and one for Spring-WS adapting. These are separate mapping and adapter interfaces unique to Spring-WS. These will do the mapping and adapting in the `WebServiceMessageReceiver` it receives the HTTP request from Spring-MVC. You can see the two phases of mapping and adapting in the gateway architecture figure.

The mapping introspects the endpoint object much like the REST mapping except here we cannot map based on the path since all requests are sent to the same path. Instead we use the `QName` of the root element of the message payload.

So the introspection again pulls out all methods which have a single JAX-B argument and return a JAX-B object and registers mappings for the `QNames` of the arguments for the endpoint object from our `EndpointInformation`. At the same time adapters are registered which can take the request, create a JAX-B unmarshaller, call the endpoint method with the resulting object, and write the response JAX-B bean back into the response output stream with a JAX-B marshaller.

WSDL

To create a WSDL for the SOAP service, the context bundle also registers a special WSDL object `DynamicWsd11Definition`. It listens for `EndpointInformation` objects for this context and merges the configured XSDs into a single master XSD in memory. It then scans the XSD for elements with the suffixes “Request” and “Response” and matches them together into the operations which make up the WSDL. Although for “RESTful” services we could receive and return any types as part of an operation, it is important that gateway endpoints use matching requests and responses, otherwise the automatic creation of the WSDL will be missing entries if there is no matching response for a request and vice versa. The WSDL is accessible per context at, e.g., `/mobiliser/transaction/Transaction.wsdl`

2.2.5 JMS

We mentioned earlier that the JMS integration uses SOAP internally. When the JMS integration is deployed, a listener is registered to collect `WebServiceMessageReceiver` objects from the OSGi registry. This works exactly like the `SybaseUrlHandlerMapping` except in this case the path name, "transaction" in our example, is used at the source JMS queue name. So a JMS listener is configured on that queue and processes incoming `TextMessage` or `BytesMessage`, converting them using the same set of Spring-WS adapters and mappings as described in the main SOAP section. Responses are written back to the reply-to queue specified by the client.

Since we cannot use the HTTP based authentication mechanisms for JMS, we have implemented a proprietary solution for authentication, when calling in through JMS

Pseudo-Basic Authentication

This is meant to match HTTP basic authentication as closely as possible. Clients should submit the username and password, concatenated together with a ":", then converted to bytes using the UTF-8 character set and then base64 encoded. So to authenticate the user `cstfull` with password `secret`, the JMS message should include a string property in the following format:

```
MOBILISER_AUTHORIZATION=Basic Y3NOZnVsbDpzZWNyZXQ=
```

Not secure, but does prevent simply viewing the username and password in clear text in the JMS broker console.

Simple Username Password Authentication

A very simple way to authenticate, but leaves the user name and password in clear text. To authenticate the user `cstfull` with password `secret`, the JMS message should include two string properties:

```
MOBILISER_PASSWORD=cstfull
MOBILISER_USERNAME=secret
```

2.2.6 TCP

Alongside the traditional HTTP interfaces to the gateway (SOAP / Restful), any endpoint can be reached through a custom TCP interface by adding an additional TCP integration bundle to the container, `com.sybase365.mobiliser.framework.gateway.tcp`. The TCP interface expects the same plain XML message accepted by the restful endpoint, but sent over TCP. Messages are framed by ending it with `\r\n` (Hex bytes `0xD 0xA`). If your XML text happens to contain this sequence anywhere inside of it (like in CDATA), you may prepend it with the byte `0x10` (0010000) to escape it. The TCP integration does not specify any contexts, therefore all services are accessible simply through the TCP port. Since some payload QNames are not unique across all contexts, certain contexts may be filtered - in Vanilla, for example, the Smartphone services are ignored.

2.2.7 OData

2.2.8 Clients

To make building clients easier, endpoints should define an interface which mirrors the contract defined in XSD. This will then be implemented by the actual endpoint which speaks to the business logic. For clients wanting to use the service in Java, these interfaces may be reused since they have no dependencies on the implementation and only need the contract classes. Clients will then integrate against this interface without knowing

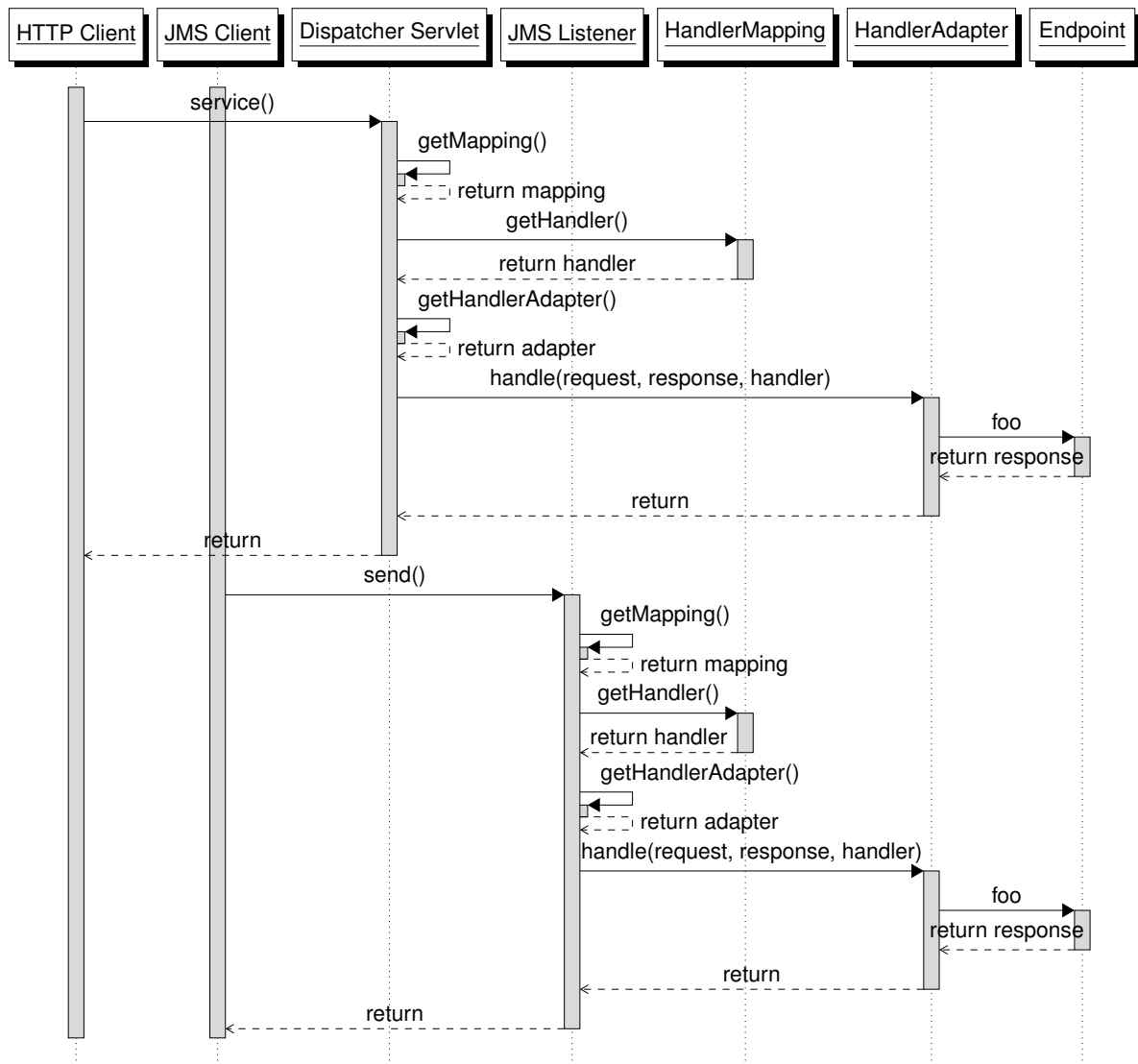


Figure 2.2: HTTP Request Sequence Diagram

the actual protocol being used. We could reconfigure the client application with another implementation of the endpoint interface and without having to change the code actually using the service.

2.3 Security

The whole security infrastructure builds on Spring Security⁷. When exposing HTTP endpoints, authentication is done through servlet filters in Spring Security. Authorisation will be described later (see 2.3.2 in detail, but is based on HTTP paths and also at the method level through JDK proxies.

2.3.1 Authentication

Filters

HTTP paths are secured using servlet filters. In a traditional Spring Security setup, a `FilterChainProxy` is statically configured to collect all the filters defined in your Spring configuration and add them to this proxy.

⁷Spring Security <http://static.springsource.org/spring-security/site/docs/3.0.x/reference/springsecurity.html>

Mobiliser extends this class to allow registering filters at runtime, which better suits the Mobiliser OSGi environment. So in standard Spring Security fashion, a `DelegatingFilterProxy` is exported through the registry as a filter for the Mobiliser servlet, which is then picked up in our case by Pax-Web. That delegate is then the extension mentioned above, `OSGiDelegatingFilterProxy`.

To add filters to this chain, we export them in a special way through the registry so it gets added to this filter chain and not directly to the servlet context. If we were to export it as a `javax.servlet.Filter` it would get picked up by PAX-Web which is not what we are trying to do. For this purpose, the security API has a simple interface `ISecurityFilter`:

```
1 public interface ISecurityFilter {
2
3     /**
4      * Returns the filter
5      *
6      * @return the filter
7      */
8     Filter getFilter();
9 }
```

Listing 2.2: `ISecurityFilter` API

Now in a traditional setup, we would create a list in our configuration and that would specify the order of the filters in the list. Since we are now configuring them at runtime, we require that all these filters implement the `Ordered` interface, which allows the deterministic definition of the filters' ordering in the list. Please note, Spring security requires that the standard filters be configured in a certain order. Please consult the Spring Security documentation for a complete overview.⁸

These filters only pull out the relevant information and then delegate the actual authentication itself to an `AuthenticationManager`. Again we are using the standard `ProviderManager` from Spring Security, adjusted to allow dynamic registration of additional `AuthenticationProviders`. These pieces actually perform the authentication. So we can authenticate against an LDAP or a flat file or a database by adding new bundles to the gateway.

Included Filters

The framework includes the following filters:

BasicAuthenticationFilter This is a standard filter provided by Spring Security. It uses the standard HTTP request header "Authorization" using the authentication scheme Basic and a base64 encoded token. The token is `username:password` Base64 encoded. If a `RememberMeServices` reference can be found in the registry, it will be notified of logins made by this filter, but more on that later.

RememberMeAuthenticationFilter We extended the standard filter provided by Spring Security to allow invalidating RememberMe cookies through a request parameter.

The filter requires a `RememberMeService` service to create, validate and invalidate `RememberMeCookies`. The framework does not provide an implementation of this, though Money Mobiliser has one which uses policies and a persistent database store.

The cookie used by Mobiliser is `MOBILISER_REMEMBER_ME_COOKIE`. If the cookie is still valid, the user is authenticated and his details are loaded from the `UserDetailsService`. Most implementations will expire the cookie after a certain idle interval but this filter can also actively expire the cookie by having the client set the request parameter `invalidate_mobiliser_remember_me=yes` and sending the cookie along with the request. This request will pass successfully and upon return the filter will invalidate the cookie. To request a cookie

⁸Custom Filters <http://static.springsource.org/spring-security/site/docs/3.1.x/reference/springsecurity-single.html#nsa-custom-filter>

from the gateway, login successfully with username and password or some other means and set the request parameter `create_mobiliser_remember_me=yes`.

AnonymousAuthenticationFilter This is a standard filter provided by Spring Security. It places a special anonymous authentication token into the security context when no other filter has authenticated the user. This practice is preferred to having an empty security context since we can configure rules based on explicit anonymity.

This is used when the actual user authentication is not done in the Mobiliser container but delegated to another system. In this case it is the business logic's obligation to authenticate the customer and it is in the external system's obligation to track e.g. number of invalid password entries.

After the authentication has happened, the business logic can instruct the framework to create a cookie, which is used for all subsequent requests to Mobiliser to authenticate the user. It is necessary that the user (Customer) gets created in Mobiliser to allow the creation of the cookie, storing auditing information, etc. This can happen when the user was authenticated for the first time. Only the first service (login) should be offered anonymously, all others can use the authentication provided by the cookie.

FilterSecurityInterceptor This is the standard filter provided by Spring Security. It enforces authorisation rules for HTTP contexts. These will be discussed in detail further on.

ExceptionTranslationFilter This is the standard filter provided by Spring Security. It translates internal Spring Security exceptions thrown further along the chain. For example, it is responsible for returning a 403 when an `AccessDeniedException` is thrown.

AuthenticationProviders

Once we have the user's name and password, as from the `BasicAuthenticationFilter`, we need to actually verify the data. The filters do not do that themselves but make use of an `AuthenticationManager` retrieved from the OSGi registry. This instance is actually a collection of `AuthenticationProviders`.

Included Providers

The framework includes the following providers:

DaoProvider This is a standard Spring security provider. It requires a `UserDetailsService` which it uses to load the users information, including locked status, expired status, his roles, and password. Further, it needs two more pieces: a `SaltSource` and a `PasswordEncoder`. The password returned from the `UserDetailsService` will typically be salted and hashed. These two implementations will transform the raw password given by the user into one that can be compared to the value returned with the user's details. Typically it will first be salted and then hashed with some algorithm.

The `UserDetailsService` service in the default configuration can only load details for the combination username and password (Vanilla identification type 5 and credential type 1). If you would like to allow authentication for other identification types, you can register an implementation of `IdIdentificationTypeResolver` with the OSGi registry. The implementation should know how to load the customer details for a given Mobiliser identification type. Since credentials may be optional for certain identifications if the user comes into Mobiliser pre-authenticated (maybe through an external system), the implementation may not return the users credential. A reusable implementation `IdTypeIdentificationTypeResolver` should fit most needs. If you just need to add MSISDN+PIN authentication so that Smartphone Mobiliser can login with that instead of username and password, just add the bundle `money.security.msisdn` to the container. It is already pre-configured for this use case.

LDAP Spring Security supports LDAP out of the box. It authenticates a user (username/password) against an LDAP instance. The authentication can either be a bind against the LDAP instance with the username / password or it can fetch the user from LDAP and compare the password. These options are configurable. Once the user is authenticated, the `AuthenticationProvider` will attempt to load the user's roles. This delegates to an `LdapAuthoritiesPopulator` instance which, in the default configuration, will populate them from an LDAP search but alternatively it can pull an instance from the registry, allowing us to use LDAP authentication but pulling the roles from elsewhere.

2.3.2 Authorisation

Authorization is also handled by Spring security. Once authentication has been successful, we have access to the roles assigned to that user and can make authorization decisions based on those.

HTTP

In the standard HTTP setup, we can assign roles⁹ to HTTP contexts. We could say clients accessing `/mycontext` must have the role `MY_PRIVILEGE`. These configurations are then enforced through the `FilterSecurityInterceptor` described previously which checks that the authenticated user may access this path. Normal non-OSGi configurations of Spring Security will define roles for HTTP paths as part of a static XML configuration. For our dynamic OSGi environment, we need a way to add new configuration when bundles are deployed or removed from the container.

`SecurityMetadataSources` are sources for security metadata in Spring security. When securing HTTP paths, an implementation can return the privileges required for a certain path. We have extended the standard implementation to allow registering new url configurations at runtime. The framework uses `ISecurityConfigs` for this purpose:

```
1 public interface ISecurityConfig {
2
3     /**
4      * Whether the roles should be ored or anded together.
5      *
6      * @return true or false
7      */
8     boolean isAnd();
9
10    /**
11     * Return the relative path.
12     *
13     * @return the path
14     */
15    String getPath();
16
17    /**
18     * String the role required for this path.
19     *
20     * @return the path.
21     */
22    String[] getRoles();
23 }
```

Listing 2.3: `ISecurityConfig` API

path The path is the path this configuration applies to relative to the Mobiliser servlet. An example would be "custom". Assuming the Mobiliser servlet is called Mobiliser, the full path is then `/mobiliser/custom`.

⁹Spring uses the concept of *roles* to determine a user's privileges. In Mobiliser language, this entity called a *privilege*, and a *role* in Mobiliser is a collection of *privileges*.

roles In Spring Security nomenclature they are called roles. These are the privileges the user must have to access this path.

and This affects whether the roles are an AND or an OR. So basically whether the use must have all of the roles or one of them to access the resource.

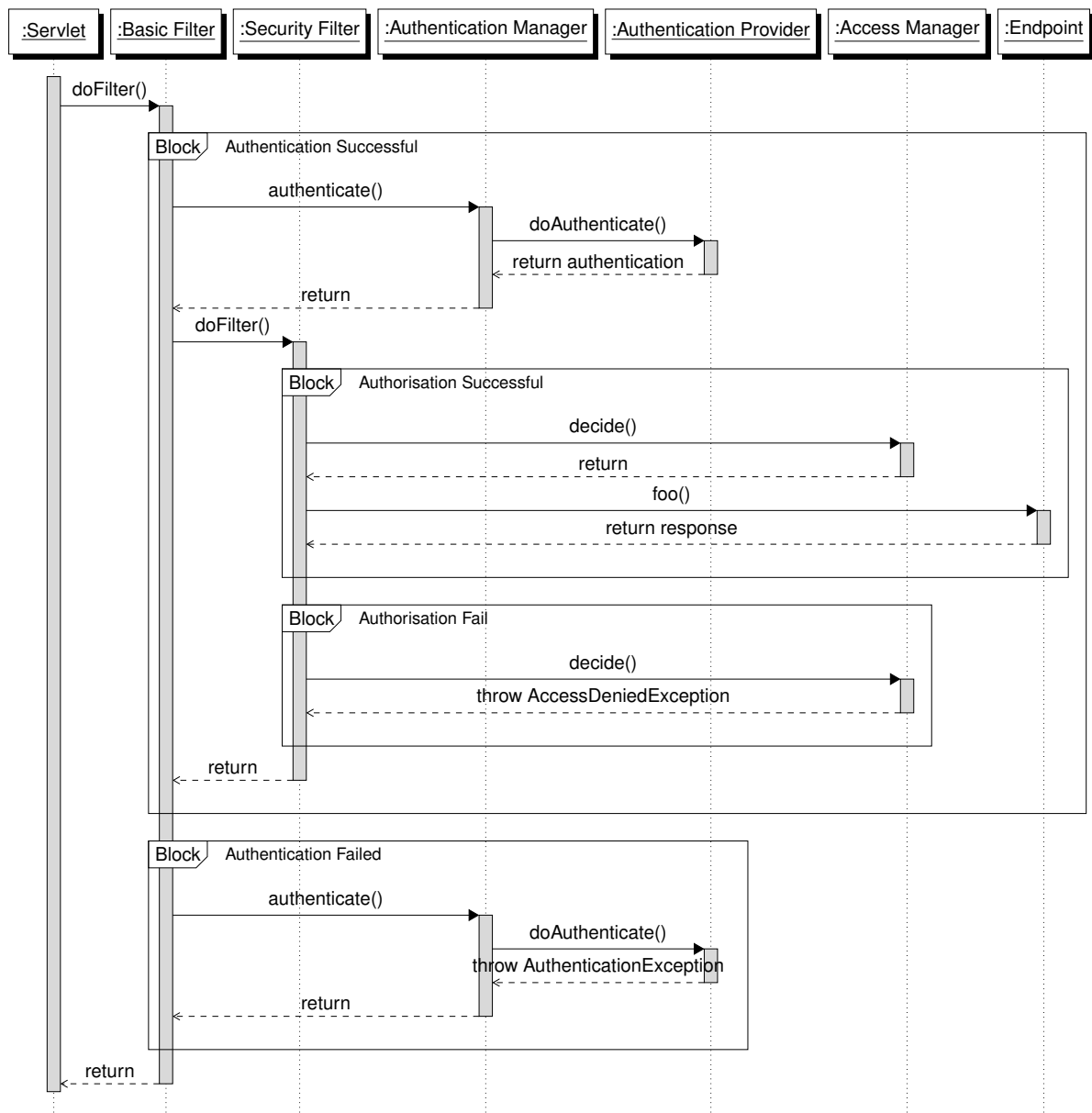


Figure 2.3: HTTP Request Security Sequence Diagram

Method Based One of the powerful features of Spring Security is method based authorisation. Spring Security exports a method interceptor which used to create a proxy of the actual endpoint which can enforce that callers have certain privileges to access a method. This is also standard Spring Security, but this section will give a quick overview.

This interceptor needs a `SecurityMetadataSource`, which basically is the configuration data for security for a given method, just as used previously when securing HTTP paths. That information is then provided to an `AccessDecisionManager` along with the current authentication object. The `AccessDecisionManager` contains a list of so called `AccessDecisionVoters`, which can vote based on the information fetched from the `SecurityMetadataSource` which is used to determine if the call is allowed to pass.

The default configuration used in Mobiliser Framework uses JSR250 annotations, which allow roles (privileges in Mobiliser nomenclature) to be configured on a method by placing an annotation with a list of roles. The access decision manager (through a `Jsr250Voter` and `Jsr250MethodSecurityMetadataSource`) then checks whether the caller has at least one of these roles, then the method call is allowed to pass.

Also included in the default configuration are the Pre and Post Authorization annotations. These are Spring specific annotations which allow code to be run before and after authorization through the Spring Expression Language. It also makes it easy to build boolean expressions for authorization.

```
1 public interface MyService {
2
3     @PreAuthorize(value = "isAnonymous() or (isRememberMe() and hasRole('WS_FOO'))"
4         )
5     FooResponse foo(FooRequest);
6 }
```

Listing 2.4: Preauthorization Annotation Example

So we only allow anonymous calls or if the user authenticated himself, then he must do it through `RememberMe` tokens and also must have the role `WS_FOO`.

Call Delegation

Another powerful feature of Spring Security we are using is the so called `RunAsManager`. This allows changing the principal and privileges of a call based on some condition. So normally we would authenticate a user and then perform some action as that user, using his privileges.

The most common scenario is to allow access to an operation a user would normally not have access to by first checking some condition. Normally only an administrator with `ROLE_X` may change customer data for example, but we could build a new service which allows a user to change his own data. So the `RunAsManager` could add the privilege required for changing the data but our new service will only allow the call to pass if the customer is changing his own information.

Another use case is, if some trusted intermediate system is invoking calls as a system user on the users behalf. The system user has all privileges required but we would like the call to appear as if the user himself is making it for auditing purposes. Here we could authorise the call as the system user but indicate in some way who initiated the call, switching the principal to that actual user in the `RunAsManager`, but keeping the privilege list as is.

Both of these are very specific scenarios that are used in Mobiliser Framework, but other setup are easily implemented through the `RunAsManager` as well.

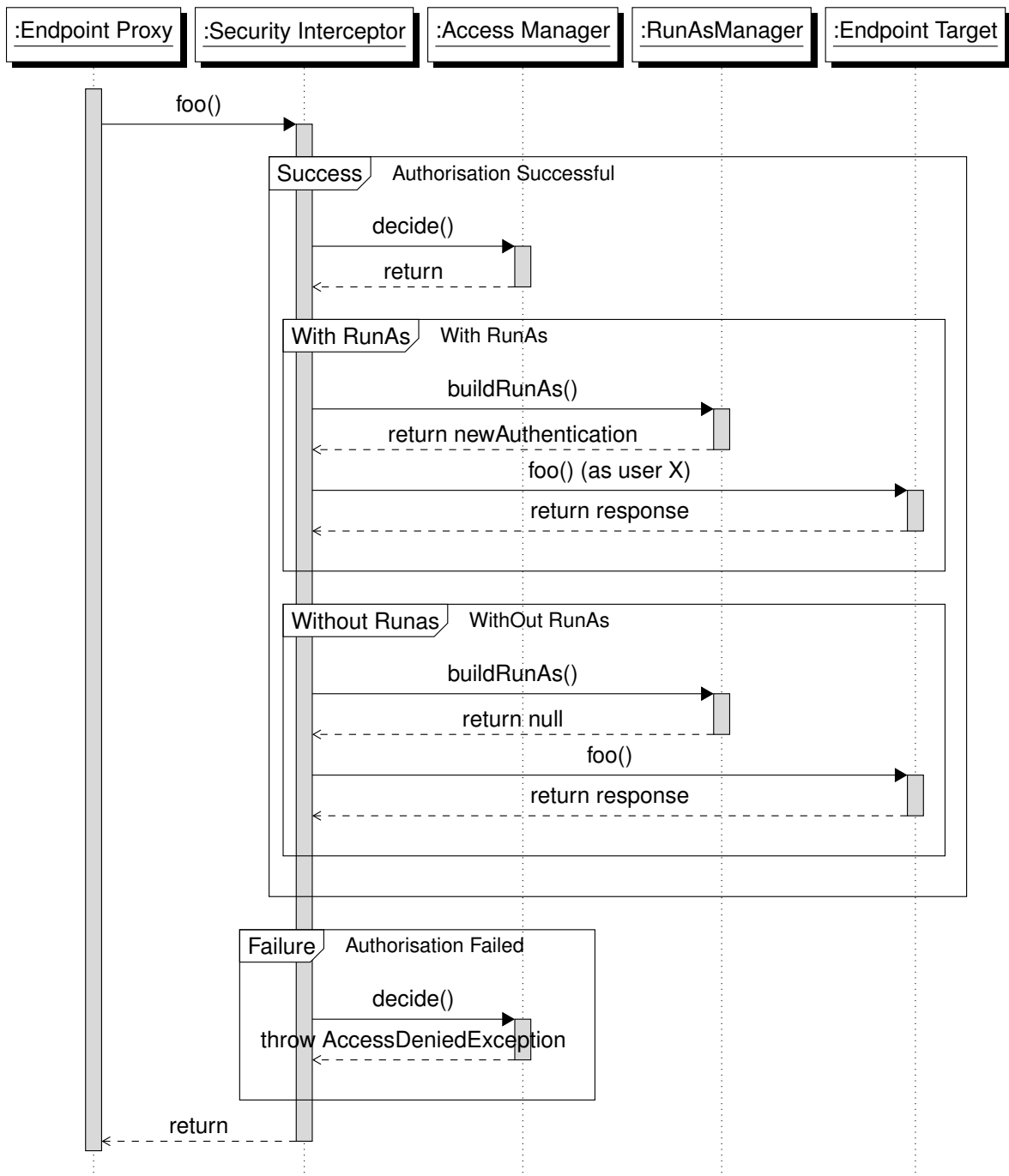


Figure 2.4: Example RunAsManager Sequence Diagram

2.4 Request Repetition

The mobiliser service gateway classifies all service into selector or modifying services. Selector calls does not change the state/data while modifying service does change the state/data. In case of an transport failure it is no problem at all to repeat any selector service request, but modifying services ¹⁰. Therefore all modifying service requests must extends the traceable request type. A traceable request always contains two mandatory attributes:

origin an identifier for the client/application.

¹⁰Example: Transfer 10 Euro from A to B. If this call is executed twice, 20 Euros are transferred

traceNumber an unique identifier for this request based on the origin.

repeat true if the client executes this request a second time, otherwise false.

Lets see how this can be used in the event of a connection shutdown or hard server failure. There are three different situation, when a request may crash

1. The request was send by the client, but not received. The client can resend the same request and the server processes it, because there is no previous request with the same origin / trace number combina-tion.
2. The request was received by the server, but the server has a hard failure during processing. Modifying request are processed in at least one transaction context. If the processing is interrupted, the transaction context is still open and therefore rolled back by the database layer. The transaction context also includes the insert of the traceable request information and after server restart, the mobiliser state is similar to that the mobiliser has never received this request. Again, the client can resend the same request and the server processes it, because there is no previous request with the same origin/trace number
3. The request is processed by the mobiliser, but the response is not received by the client

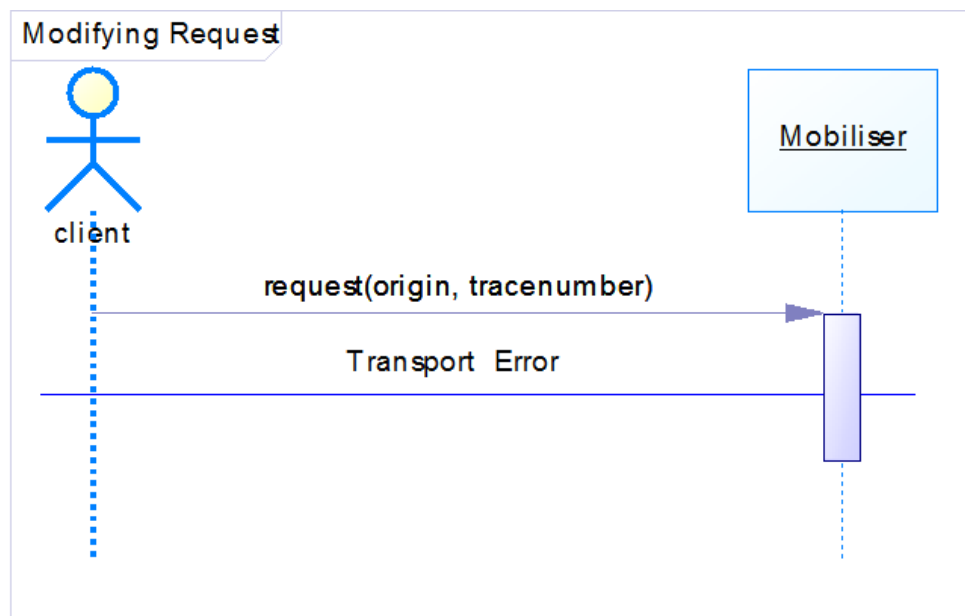


Figure 2.5: Request Interruption

Again, the client resend the same request but now the mobiliser will return the status code 51 - repeating request. The client now sets the attribute repeat to true and issue the same request again. The mobiliser recognizes this, retrieves the original request from the database layer and sends it back to the client.

Traceable request responses are stored per default for 72 hours and a cleanup job removes old entries from the database.

The following error codes may be returned by traceable requests

- 51 trace number is not unique, but repeat flag missing
- 52 cannot return cached message, because original request is still in progress
- 53 wrong user for repeat, the original request came from another client
- 54 repeat flag set, but no message found for given trace number
- 55 no response cached for trace number (this should only happen through severe system crashes)

Chapter 3

Messaging Framework

Mobiliser provides a framework for sending out correspondence to customers. Standard implementations are able to send things like SMS, USSD or emails and customized channels can be added at any time.

3.1 Message Gateway

The message gateway provides a single point of entry for sending correspondence to customers. It takes care of multi-language issues, message logging, protocol abstraction, and routing.

3.1.1 Messages

The message types are concrete implementations for each outgoing type, but all messages share the following common attributes:

recipient the format will match the type of message being sent

sender the format will match the type of message being sent

content some types will place restrictions on what the actual content of the message is, or how many parts or attachments are allowed

external id allows channels to associate an external id with this message.

An email type might also have a subject attribute, something that an SMS won't have. Other message types might allow assigning a priority or some other flags.

Mail Jobs Messages submitted through the message gateway are saved to a jobs table tracking the status and content of the job. Normally a message is sent immediately, but if sending fails, this table also serves as the queue, allowing messages to be retried. In a multi-node Mobiliser setup, one node may be configured in such a way that no messages are sent from it. The gateway will then queue all messages to the table, which in turn will be picked up and dispatched by the other node.

Logging Messages submitted through the gateway are logged to a message log table and optionally associated with a customer identifier. To prevent sensitive information from being logged, the messages may be marked as confidential which prevents the message text from being saved in clear text. Either no body for the message is logged or an alternative, a log able text may be provided for the message.

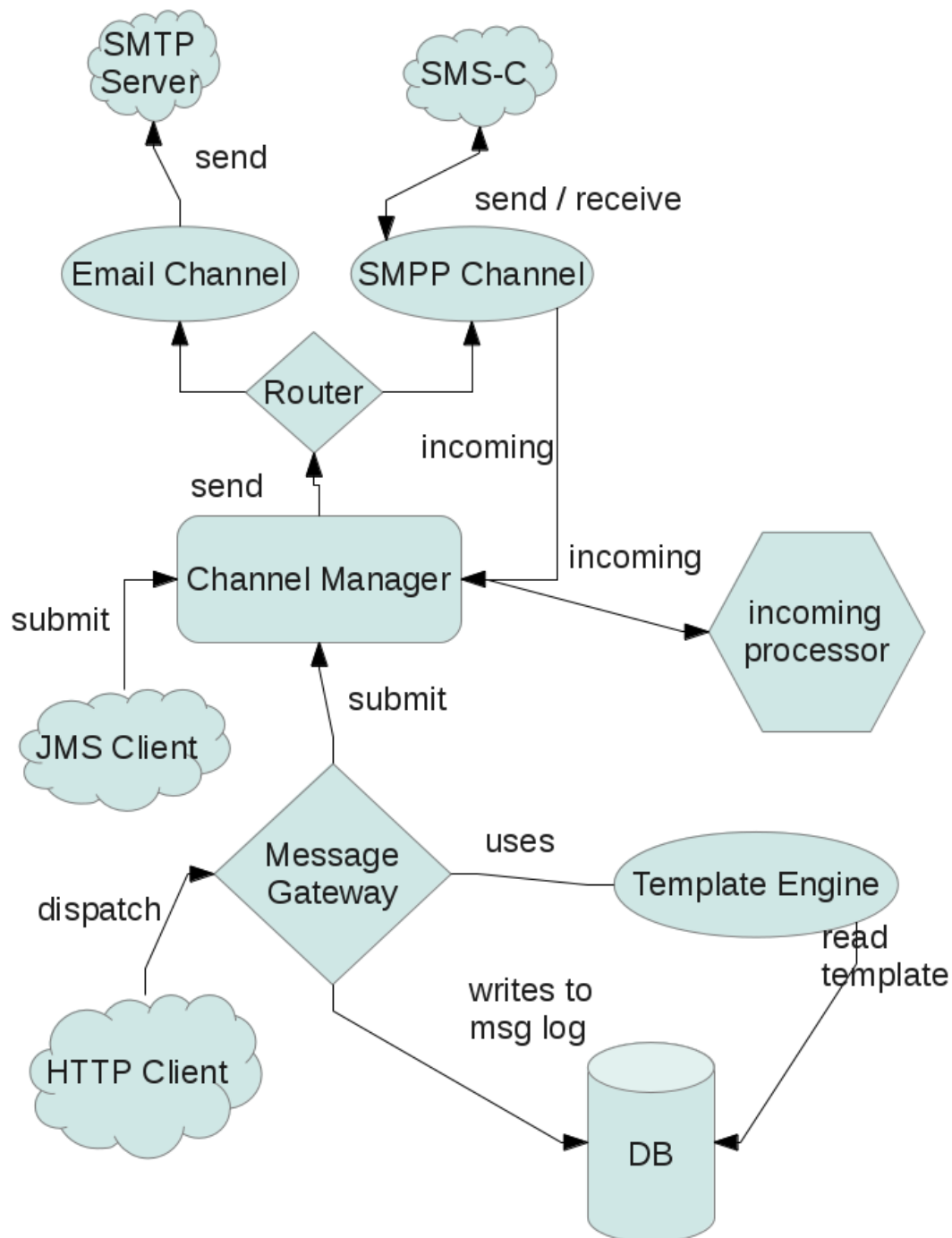


Figure 3.1: Messaging Framework Architecture

Encryption The mail jobs mentioned earlier are encrypted to avoid exposing any confidential information in the message.

3.2 Templating Engine

A templating engine was implemented to support multi-language correspondence. Clients of the engine will refer to a configured correspondence by template name and locale. The engine will then choose the template which most closely matches the supplied user locale. Please see the *Mobiliser Framework Development Guide*

for more details. Clients of the template engine supply the necessary parameters to fill the placeholders in the template and are returned the message object which can be fed into channel manager.

Template A template consists a the text of the message, the type of message, the sender, the associated locale and 0 or more attachments. Templates allow defining place-holders so that clients of the engine may dynamically build the text of the template. Please see the *Mobiliser Framework Development Guide* for more details. An example would be a transaction confirmation including the amount and date of the transaction. Clients of engine would then supply the recipient, the template name, the customer locale, and the template parameters, which in this case would be the amount and date.

3.3 Channel Manager

Channel Manager provides the abstraction between the message gateway and the actual protocol implementation used to send the message to the customer. The manager, as the name suggests, manages the channels in the system and routes messages to the correct outgoing channel which then does the actual sending.

Channel A channel is the basic component used by channel manager. It implements a protocol for sending message, i.e. SMTP or SMPP. Channels are registered with channel manager and publish which types of messages they support. So an SMS cannot be sent through an email channel, since we would be unable to effectively route the message with an MSISDN. Channels can also receive messages for protocols that support two way messaging, such as SMPP. The channel then communicates back to channel manager through a callback interface, delegating the further routing of the message to channel manager. Systems which should receive these messages register themselves with channel manager as a listener and then consume these incoming messages.

Routing Multiple channels may be registered with the same id. To resolve which channel should be used, any number of `ChannelRouters` may be registered. An example for this would be if messages for certain subscribers should be sent through a certain SMS-C. If no such router can be found, channel manager will simply choose the first channel capable of sending that message type.

JMS In most cases clients will directly invoke the message gateway API to submit messages. As an alternative, we have a JMS capable channel manager which allows message objects to be directly submitted bypassing the gateway. When using this interface, clients must create the message objects themselves.

3.3.1 Overview

The Channel Manager is used for SMS and USSD communication for Brand Mobiliser through a JMS interface too. The actual protocol implementation is done as a channel that is deployed into the Channel Manager and not directly in the message gateway or Brand Mobiliser. The communication between Channel Manager and Brand Mobiliser and between Channel Manager and the message gateway is via ActiveMQ queues or through direct OSGi service communication, dependent on how the containers are deployed. The channels themselves not aware of anything JMS related, though each has a channel ID managed in the preferences. Nevertheless it helps to understand the internal mechanism that is used for the communication between Channel Manager and Brand Mobiliser.

Mobile Originated (MO) sessions only use the Brand Mobiliser InQ and an additional temporary queue that is established for each channel instance.

The channel always sends messages to the configured "inQ" to Brand Mobiliser and listens on the temporary queue for messages from the Brand Mobiliser or from the message gateway. Optionally it can also listen on a separate "outQ" for messages that are triggered through Brand Mobiliser (aka: push messages).

3.3.2 API

This section describes the main interfaces in the Channel Manager.

Channel

`com.sybase365.Mobiliser.util.messaging.channelmanager.api.Channel` is the base interface for all channels. It does not define any functionality, only that a channel has an identifier. In a JMS setup, this is the name of the queue used by Brand for push messages to this channel.

```
1 public interface Channel {
2
3     /**
4      * This channel's id.
5      *
6      * @return the channel's id
7      */
8     String getChannelId();
9
10 }
```

Listing 3.1: Java Channel API

SendChannel

`com.sybase365.Mobiliser.util.messaging.channelmanager.api.SendChannel` is the base interface for channels which can send messages. These channels are capable of receiving a message object from `ChannelManager` and sending it to the client in some way. A good example of this would be the `EmailChannel`, which is capable of sending e-mails to clients.

```
1 public interface SendChannel extends Channel {
2
3     /**
4      * Sends the message using this channel. You should first check whether this
5      * channel is capable of sending the message with {@link #supports(Message)}
6      *
7      * @param message
8      *         the message to send
9      * @throws ChannelException
10      *         if the message cannot be sent
11      */
12     void send(final Message message) throws ChannelException;
13
14     /**
15      * Returns whether this channel supports the given message.
16      *
17      * @param message
18      *         the message
19      * @return true or false
20      */
21     boolean supports(final Message message);
22 }
```

Listing 3.2: Java SendChannel API

`com.sybase365.mobiliser.util.messaging.channelmanager.api.AsynchronousSendReceiveChannel` is a further extension of a `SendChannel`, in that it can also at any time receive messages. The `SmppChannel` would be a classic example, since it can send SMS to the customer at anytime, but can also receive messages from the SMS-C also.


```

1 public interface AsynchronousSendReceiveChannel extends SendChannel {
2
3     /**
4      * Set the receive callback through which this channel can push out incoming
5      * messages to some consumer.
6      *
7      * @param receiveCallback
8      *       the receive callback
9      */
10    void setReceiveCallback(AsynchronousChannelReceiveCallback receiveCallback);
11
12 }

```

Listing 3.3: Java AsynchronousSendReceiveChannel API

SynchronousReceiveChannel

`com.sybase365.mobiliser.util.messaging.channelmanager.api.SynchronousReceiveChannel` is a channel capable of receiving messages, getting a response for it, and then returning that response all in one session. A good example is any channel using HTTP as its protocol. A client would send a message to the channel through HTTP, the message is forwarded to `ChannelManager` which eventually gets a response, this response is handed back off to the channel, which then gets written back to the client through HTTP. `SynchronousReceiveChannels` cannot send messages by themselves, only responses to previously received messages.

```

1 public interface SynchronousReceiveChannel extends Channel {
2
3     /**
4      * Set the receive callback through which this channel can push out incoming
5      * message to some consumer and receive a response.
6      *
7      * @param receiveCallback
8      *       the receive callback
9      */
10    void setReceiveCallback(
11        final SynchronousChannelReceiveCallback receiveCallback);
12
13 }

```

Listing 3.4: Java SynchronousReceiveChannel API

HTTP Channel

`com.sybase365.mobiliser.util.messaging.channelmanager.api.HttpChannels` are directly accessible through HTTP, so they include an additional method to process incoming HTTP requests. They also include an additional method to define the path used to reach them and is relative to the `ChannelManager` path itself.

```

1 public interface HttpChannel extends SynchronousReceiveChannel {
2     /**
3      * This defines the URL supplement that is used by this channel. All HTTP
4      * requests starting with this URL supplement will be forwarded to this
5      * channel.
6      *
7      *
8      * @return url supplement
9      */
10    String getUrlSupplement();
11
12     /**
13      * Similar to the doPost method from the HttpServlet this must process the

```

```

14     * Http request/response.
15     *
16     * @param request
17     * @param response
18     * @throws ServletException
19     * @throws IOException
20     */
21     void processRequest(final HttpServletRequest request,
22                         final HttpServletResponse response) throws ServletException,
23                         IOException;
24
25 }

```

Listing 3.5: Java HttpChannel API

AcknowledgingChannel

`com.sybase365.mobiliser.util.messaging.channelmanager.api.AcknowledgingChannel` is a channel which is capable of acknowledging previously sent messages. Not all protocols will support this, but this will allow the channel to notify `ChannelManager` that a message previously sent by this channel has been successfully received by the customer.

```

1 public interface AcknowledgingChannel extends Channel {
2     /**
3      * Sets the acknowledge callback this channel should use to
4      * acknowledge previously sent messages.
5      *
6      * @param receiveCallback
7      *         the callback.
8      */
9     void setAcknowledgeCallback(final ChannelAcknowledgeCallback receiveCallback)
10    ;
11 }

```

Listing 3.6: Java AcknowledgingChannel API

Callbacks

Depending on whether the channel is synchronous or asynchronous, it may make use of receive callbacks.

SynchronousChannelReceiveCallback This channel will pass of the message and receive a standard Java `Future` which can be used to get the response. The channel may make use of `get()` or `get(long timeout, TimeUnit unit)` to retrieve the response object, noting that null may be returned if processing fails or takes too long. The channel id refers to the channel's id, the destination id to the id of the destination. It will most likely be configured into the channel through the preferences. In a JMS setup, the destination id will be the name of the queue used for sending the message to Brand.

```

1 public interface SynchronousChannelReceiveCallback extends
2     ChannelReceiveCallback {
3
4     /**
5      *
6      * @param message
7      *         the received message
8      * @param channelId
9      *         the channel id which received the message
10     * @param destinationId
11     *         the destination id

```

```

12     * @return message which is the response to the message received
13     */
14     Future<Message> receiveAndRespondMessage(final Message message,
15         final String channelId, final String destinationId);
16 }

```

Listing 3.7: Java SynchronousChannelReceiveCallback API

AsynchronousChannelReceiveCallback This is basically a fire-and-forget interface where the channel hands off the incoming message and forgets about it. The parameters here have the same meaning as in the synchronous example.

```

1     /**
2     * @param message
3     *         the received message
4     * @param destinationId
5     *         the destination id for this message
6     * @param channelId
7     *         the channel which received the message
8     */
9     void receiveMessage(Message message, final String channelId,
10         final String destinationId);

```

Listing 3.8: Java AsynchronousChannelReceiveCallback API

ChannelAcknowledgeCallback This callback is used to asynchronously acknowledge messages.

```

1     /**
2     * Notify that we received an acknowledgement for the message with the given
3     * id.
4     *
5     * @param messageId
6     *         the message id
7     * @param channelId
8     *         the channel which received the acknowledgement
9     * @param originalSendDate
10    *         the date the original message was sent. may be null if the
11    *         information is not available to the channel
12    */
13    void acknowledge(final String messageId, final String channelId,
14        final Date originalSendDate);

```

Listing 3.9: Java ChannelAcknowledgeCallback API

This original send date may be useful for the implementation if message ids can repeat after certain intervals. This allows the implementation to try to locate the correct message to be acknowledged.

IMessageRouter

ChannelManager will listen for new channel instances and will make use of them when sending outgoing messages. An IMessageRouter allows choosing between multiple channels registered with the same id. The default router will simply choose the first channel which returns true from supports, but a custom router could choose based on some other criterion.

```

1     /**
2     * Choose the best channel for the given message.
3     *
4     * @param channels

```

```
5      *           the channels
6      * @param message
7      *           the message
8      * @return the channel, or null if no channel matches
9      */
10     SendChannel chooseChannel(final List<SendChannel> channels,
11                               final Message message);
```

Listing 3.10: IMessageRouter API

Chapter 4

Preferences

Preferences are the standard mechanism for application configuration in Mobiliser. They are mainly used to manage operating level configuration data such as URLs, user name, timeouts, retries when communicating with other systems, or thread/object pool size and the like. In other situations, they are used for business logic configuration like names of message templates or the customer type id used for creating new customers. They replace the use of properties files seen in many other projects or components.

4.1 Structure

If you are familiar with the standard `java.util.prefs.Preferences`, then you already understand most of the design of the Mobiliser preferences. Preferences are basically named trees with maps of key-value pairs at each leaf. Configuration is done by looking up a named leaf in a particular tree and inspecting the key-value pairs there. The standard convention is to use the object's class name to find the leaf with the configuration. For example, when configuring an instance of `com.sybase365.foo.FooImpl`, we would look up the leaf `/com/sybase365/foo/FooImpl/` and find the set of properties there.

The standard Java Preferences have the notion of system preferences and user preferences, where system preferences are global while the user preferences are looked up based on the user name of the user running the JVM. Our preference trees have names like the user preferences, but a configuration value tells the application which tree to use. We also chose to implement our own independent preferences since the standard preferences are not really compatible with OSGi because of the static way the tree is initialized. This allowed us more flexibility and also avoided class loading issues with the preferences. While there is a mirrored implementation for OSGi, by using our own interfaces, we could implement our preferences and use them in the OSGi container and at the same time use them in Tomcat for the web-front ends.

4.2 API

The two main interfaces are:

- `com.sybase365.mobiliser.util.prefs.api.IPreferencesService`
- `com.sybase365.mobiliser.util.prefs.api.IPreferences`

The `IPreferences` interface offers the same read operations as the standard preferences, but does not allow changing them through this interface. This reduces complexity in the implementation, with refreshing becoming almost a simple replacement operation. It provides `getXX(String, String)` methods for most data types and also provides a way to force a refresh of the node and its children.

The `IPreferencesService` interface is the lookup interface, used to get the root node. This is an infrastructure interface, and should have no direct use during normal development.

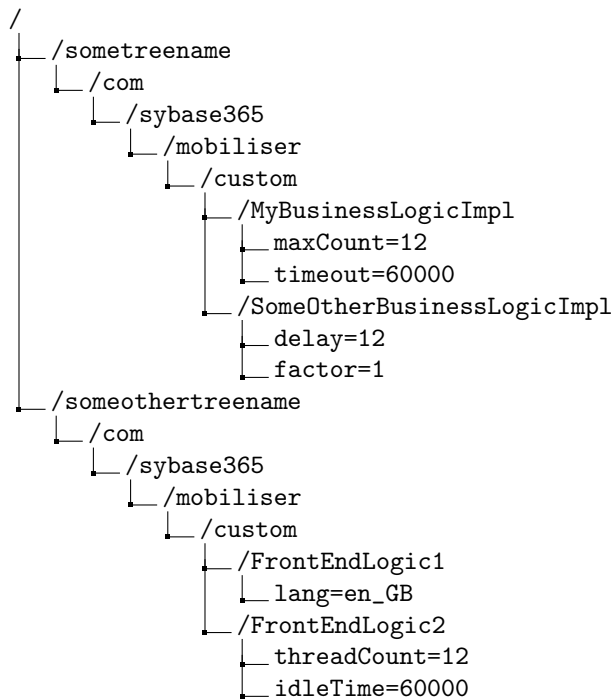


Figure 4.1: Example Preferences Trees

4.3 Access

The preference interface is very simple and only allows read access to the tree. Clients may request that a node be refreshed from its source, but the API does not define any write operations making the implementation very simple.

The values held in the preferences may have an implicit meaning, but from an API point of view, they are just string key value pairs.

To maintain a clean design, each piece of the preferences is responsible for a single thing. Additional functionality has been added to the plain string key value pairs by implementing interceptors in the preferences. The actual nodes hold the data in maps and based upon an injected `IBackingStore` can fetch the actual data from some backing store.

4.3.1 IBackingStore

A backing store is configured at runtime which the nodes then use to fetch the preferences data. This implementation can be plugged in dynamically, which allows us to use the same preference implementation in an OSGi container and in Tomcat by configuring different backing stores.

4.3.2 Interceptors

Interceptors wrap the base preferences node, implementing additional functionality - things like encryption, logging, property replacement.

Encryption We have implemented an encryption layer on top which allows preference values to be encrypted in the backing store and then dynamically decrypted when the values are read. The values are held in memory in encrypted form and only decrypted when requested. Encryption algorithms may be configured at runtime.

Preferences are encrypted and decrypted by an implementation of `IPreferencesEncryptionStrategy`. An `IPreferencesEncryptionManager` manages all known strategies currently known to the container. Given a preferences

value, it can determine if the value has been encrypted. The current implementation does this by prefixing encrypted values with `${SCHEME}`, where `scheme` is something like `AES-256` or `3DES`. Matching each `IPreferencesEncryptionStrategy` is a `IPreferencesEncryptionStrategyFactory` which knows this prefix and how to create the strategies.

```
1 public interface IEncryptionStrategyFactory {
2
3     String getPrefix();
4
5     boolean supportsType(final String type);
6
7     IPreferencesEncryptionStrategy createEncryptionStrategy(
8         final Properties properties);
9 }
10
11 public interface IPreferencesEncryptionStrategy {
12
13     String encryptValue(final String preference);
14
15     String decryptValue(final String encryptedPreference);
16
17 }
```

Listing 4.1: Preferences Encryption APIs

The decryption interceptor inspects preferences values and if they have an encryption prefix, they are decrypted on the fly using the correct strategy retrieved from the encryption manager before being returned.

System Properties In a similar fashion, we have added a layer to allow embedding system properties as part of preference values. This is useful for having the same base value for all nodes, but then adapting that for each individual node at the system property level. Another interceptor checks for embedded system properties in the form `${backend.url}`. These are then dynamically replaced with the matching value from the system properties before being returned. Then you have the placeholder in the preferences but the actual value is in one place. If it changes, you then just change it in one place, not in every single preferences value.

4.4 Refresh and Events

Preferences values can be changed at runtime. This is done by changing the values in the backing store. The preferences are initialized with a refresh thread which reads all preferences values from the backing store at regular intervals and updates the values held in the nodes.

To avoid having to poll for new values, client code can register listeners to receive events when preferences values are changed.

```
1 public interface IPreferenceChangeListener {
2     void preferenceChanged(final IPreferenceChangeEvent evt);
3 }
```

Listing 4.2: Preference Listener API

These events are triggered when a value is added to, remove from, or changed in a node.

```
1 public interface IPreferencesNodeChangeListener {
2     void childAdded(final IPreferencesNodeChangeEvent evt);
3
4     void childRemoved(final IPreferencesNodeChangeEvent evt);
5 }
```

Listing 4.3: Preference Node Listener API

These events are triggered when a completely new subnode is added to another node or if the whole subnode is removed.

Using these features allows us to create complex objects based on preferences configuration and then register a listener to recreate the object with the new values if they change.

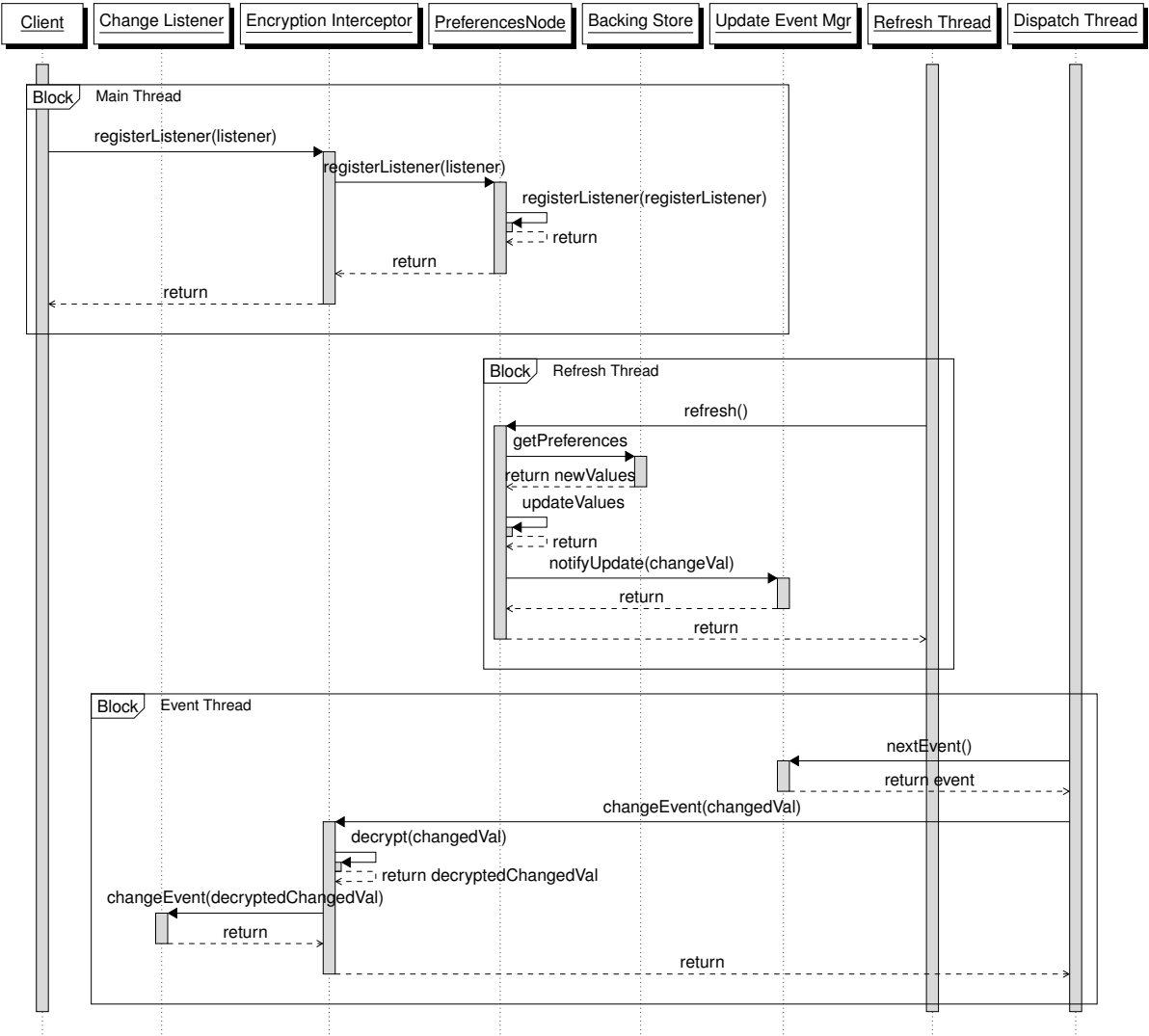


Figure 4.2: Sequence Diagram Preferences Refresh + Listener

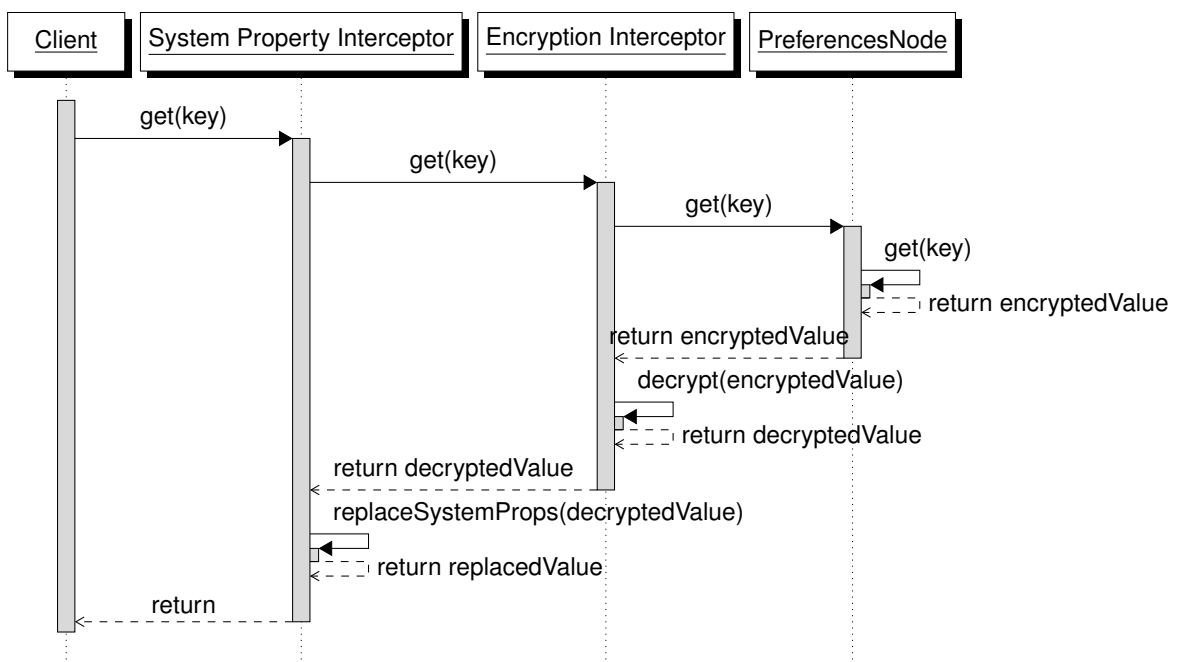


Figure 4.3: Sequence Diagram Get Preferences Value

Chapter 5

Persistence

One of the major components in the Mobiliser Framework is the persistence layer. The persistence layer is responsible for storing and retrieving information into and from a database system and abstracts the actual operations from the other Mobiliser components.

The persistence layer follows the classical data access objects (DAO) pattern.

1. Each database table is modeled in a JPA compliant bean.
2. For each bean, the correlated interface defines the DAO API, i.e. the different search/update/insert/delete methods, which are applicable to the specific bean/table.
3. A concrete implementation of the DAO API provides physical access to the database.

This pattern allows to hide any specifics of the database access in the DAO implementation classes. Any other Mobiliser business logic does not depend on this, but is implemented only depending on the API definition. Mobiliser Framework comes with an Hibernate implementation of the DAO APIs, but this can be replaced with any other persistence framework—not affecting business logic code.

Mobiliser supports the following database management systems:

- Sybase Adaptive Server Enterprise 15.5
- IBM DB2 UDB 9.7.4 Common Server
- PostgreSQL 9
- Oracle 10gR2

5.1 Data Model Beans

The data model beans of Mobiliser use only JPA compatible annotations to build up the entity bean class. It is not valid to use, e.g., Hibernate (or any other persistence framework) specific annotations (although the reference implementation uses Hibernate as the persistence framework).

5.2 Data Access Object API

To abstract the use of a direct database connection, a DAO API is introduced. All database access and object manipulations reaching from simple loading an entity to complex select or update queries have to be done by using a DAO object. These DAOs hide the direct database connection, session and transaction, as well as the

concrete implementation (*SQL dialect and database system*) of the queries. This makes handling database objects much more transparent and interchangeable from a business logic point of view.

In order to achieve this, for each JPA bean, there exists a corresponding DAO interface to define the DAO's functionality and to hide the actual implementation. Each DAO interface extends one of the following base interfaces, which provide a predefined set of methods which each concrete DAO implementation has to provide. There is a strong relationship between the base class that is extended by the JPA bean, and the interface you would have to use as a basis for the DAO interface.

Base Interface	Description
BaseDAO	Has to be used if writing a DAO for a JPA bean extending DbEntry
UpdatableDAO	Has to be used if writing a DAO for a JPA bean extending UpdatableDbEntry
IdDAO	Has to be used if writing a DAO for a JPA bean extending IdEntry or NamedLookupEntry
GeneratedIdDAO	Has to be used if writing a DAO for a JPA bean extending GeneratedIdEntry
NoneUpdatableGeneratedId	Has to be used if writing a DAO for a JPA bean extending NoneUpdatableGeneratedIdEntry
CompositeIdDAO	Has to be used if writing a DAO for a JPA bean extending CompositeIdEntry
ICleanableEntriesDao	Can be implemented if the entries of the related table can be automatically deleted after a certain period of time

Table 5.1: Base DAO interfaces

To ensure a certain level of type safety the DAO API makes use of generics. All interfaces are parameterized by the entity class and the entity id. When extending the `GeneratedIdDAO`, the id is already fixed to Long.

DAOs extending one of the base DAO interfaces automatically have methods defined to create a new instance, save/update/delete an entity, get an entity by id, and a few more. All other functionality any DAO should provide is implemented in "custom" query methods, which are unique to this particular DAO (and also to its child classes).

If the optional interface `ICleanableEntriesDao` is extended a `deleteEntriesOlderThan` method needs to be implemented defining how entries older than a specified threshold can be removed safely.

The `dao-api` bundle also defines a generic DAO factory interface, defining getter methods for each supported DAO interface. This way each DAO factory implementation is forced to implement a getter to access the DAO interface implementation. The naming convention follows the standard Java naming convention for getter methods.

5.3 Data Access Object Implementation

The Mobiliser Framework today supports Hibernate as the only persistence framework. The Hibernate persistence bundles provide the `Hibernate SessionFactory` and `Spring PlatformTransactionManager` consistently as OSGi services, such that implementations can rely on them. The actual Hibernate beans are discovered dynamically through the OSGi service registry and are defined by implementations of `PersistenceServiceProvider`.

5.3.1 Persistence Service Provider

Each JPA model bean is added to the list of model beans returned by the implementation of the `PersistenceServiceProvider` interface. This is used to construct one common (Hibernate) Session Factory that is aware of all Model beans that are deployed into the container.

A `PersistenceServiceProvider` defines

- The list of Hibernate beans,
- a cache configuration file name
- the required database setup version.

The default database setup version check simply validates against the `MOB_VERSIONS` table. The Hibernate factory needs the list of persistent bean classes and the cache strategy. This is done by implementing the `com.sybase365.mobiliser.framework.persistence.hibernate.sessionfactory.api.PersistenceServiceProvider` interface.

The `getPersistenceClasses()` method returns the list of provided classes and their cache strategy (see Caching for details on defining the cache strategy).

Each time a new `PersistenceServiceProvider` is registered with OSGi the current session factory is automatically dropped and re-build, including the newly registered model beans. This behavior is necessary since all model beans must be “known” upon creation of the session factory. A “lazy” or “delayed” registration of new model beans, or model beans the current session factory is unaware of, is not possible.

5.3.2 DAO Factory

The Hibernate DAO factory is also located in the `dao-hibernate` bundle. It implements the `DAOFactory` interface and offers getter methods for each DAO interface.

Each DAO implementation class is instantiated and injected via Spring into the Hibernate DAO factory.

If you inspect the `bundle-context-osgi.xml` you will notice that the `sessionFactory` is imported from the OSGi registry (and added to each DAO implementation). The reference Hibernate DAO factory itself is exported as an OSGi service and can be consumed by any bundle in the container through the DAO factory interface. This way, any using bundle is agnostic to the underlying DAO implementation and works purely on the defined DAO interfaces and DAO model entity beans. The Mobiliser framework bundles not only export the session factory that is used by the DAO implementation classes, but they also export the accompanying transaction manager.

Chapter 6

Reports

Mobiliser uses the “Crystal Reports Framework” for generation and display of reports. Reports are automatically discovered from the hard disk and available through the Mobiliser reporting API as well as through the Mobiliser front-end.

6.1 Introduction

The embedded reporting scenario uses the Java Reporting Component (JRC) and Crystal Reports Viewers Java SDK to enable users to view and export reports in a web browser. They provide the functionality required to create and customize a report viewer object, process the report, and then render the report in DHTML. The JRC keeps report processing completely internal to the Java application server. The JRC allows you to process Crystal Reports report (.rpt) files within the application itself, without having to rely on external report servers.

Note: The JRC is simply a logical set of JAR files. This scenario is labeled embedded reporting because using the JRC requires the report engine JAR files to be embedded within your Java application.

Report Engine: In the embedded reporting scenario, the Java Reporting Component (JRC) is the report engine. The JRC processes report requests from the viewer and exposes an object model that allows developers to interact with a report through code. With the JRC, all processing is done within the Java application server.

The ReportClientDocument Object Model: To interact with the report through code, the JRC provides a ReportClientDocument object model. This object model encapsulates the Crystal Reports report (.rpt) file and provides a runtime instance of the report including its data. When you are ready to view the report, the ReportClientDocument object has a report source property that you can pass to the viewer for display.

Note: Although the JRC can modify the ReportClientDocument instance at runtime, these modifications are not persisted back to the Crystal Reports report (.rpt) file. Only the Crystal Enterprise Report Application Server has the ability to persist runtime modifications. An exception to this rule is the data source location. The JRC can modify the data source location in a Crystal Reports report (.rpt) file at runtime, and then persist this change back to the report file.

The ReportClientDocument object model exposed by the JRC is a subset of the ReportClientDocument object model that is exposed by the Crystal Enterprise Report Application Server. This common architecture simplifies application migration from embedded to enterprise reporting.

The Report Source: The viewers use a report source rather than a report object model to interact with a report. A report source enables the viewers and the engine to communicate more efficiently during high demands for report processing. The ReportClientDocument object has a report source property that you can pass to the viewer for display.

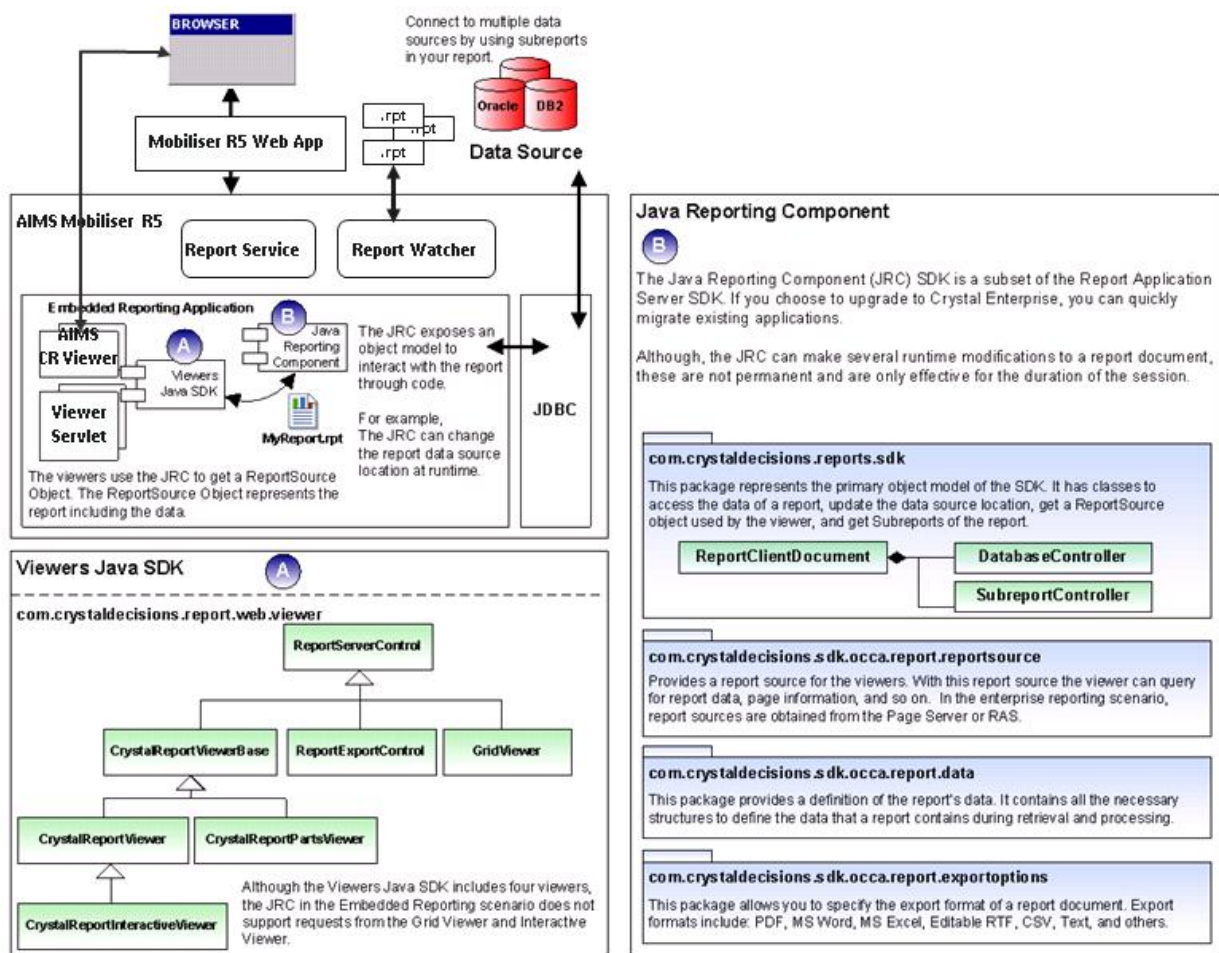


Figure 6.1: The Crystal Reports Embedded Reporting Architecture

Report Viewers: In the embedded reporting scenario, the Crystal Reports Viewers Java SDK provides the report viewers. The Crystal Reports Viewers Java SDK allows you to build customized report viewers to display reports as Dynamic HTML (DHTML). You can include the viewers in your JavaServer Pages (JSPs) and manipulate them through code to create the presentation that you want. This enables you to minimize hand coding the presentation of data.

The Crystal Reports Viewers Java SDK also supports exporting (PDF, RTF, and Crystal Reports formats), printing, displaying multiple viewers in the same page, and both automatic and developer-specified prompting for database logins or parameter information.

Report Designer: The Crystal Reports report (.rpt) file specifies the data source and data presentation information. These rpt files are binary only and are created/updated using the Crystal Reports Designer product. As the JRC component only support a subset of the full Crystal Reports functionality, it is recommended to use Crystal Reports Designer 2008 for maximum compatibility. The Crystal Reports for Eclipse(CR4E) 2.0 product can also produce report files for JRC to render; however, it does not support defining SQL queries in the report. For more information, see <http://www.sdn.sap.com/irj/sdn/crystalreports-java>

6.2 Mobiliser Reporting Framework

We use Crystal Reports for both, designing and rendering the reports. Report design is an implementation task, and reports are simply added to the Mobiliser installation by dropping them into the right place on the hard disk. Mobiliser will discover these reports, get them validated through the Crystal Reports engine, and

publish them through the reporting API.

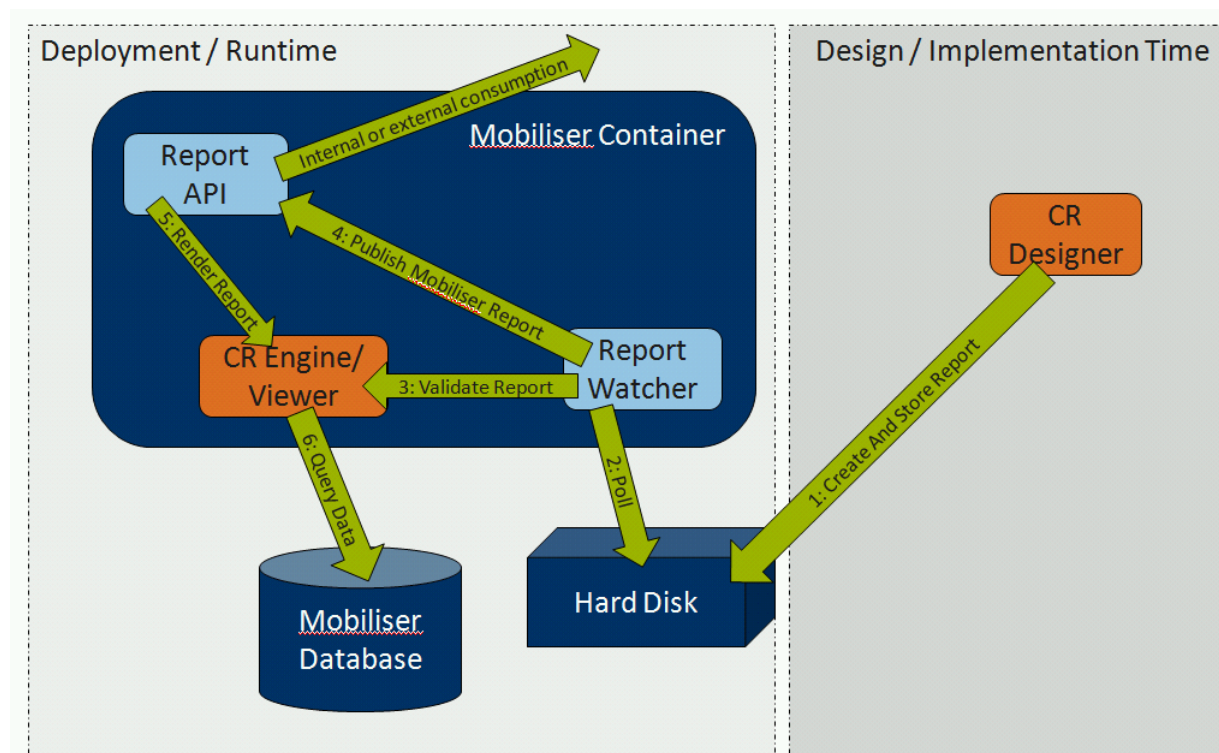


Figure 6.2: Mobiliser Reporting Architecture

Once a rendered report instance is requested, the embedded Crystal Reports engine is used to produce either a static output or a dynamic web-based view. As part of the report rendering, the required data is pulled out of the Mobiliser database.

6.3 On-line Reports (Ad-hoc Reports)

On-line reports may be requested and viewed by the Mobiliser front-end. The parameters, which need to be supplied for the specific report to be generated are retrieved from the rpt file by the report watcher when scanning for available reports. For each parameter the front-end will render a corresponding input field matching the type of the parameter e.g. datepicker field for dates, drop-down for "restricted" value sets. After the user provides the required parameters, a web service request is sent returning the report view generated by the server which is displayed by the front-end.

6.4 Asynchronous Reports

If generation of the report is known to take a long time, reports may be requested to be generated asynchronously. These reports are stored on the server and may be downloaded by the User which requested the report once the report has been created.

Asynchronous Reports may also be created by defining schedules for the reports (e.g. once a day, once a month, on the last Friday of every Quarter etc.) using the cron format. Defining such a schedule may be done in the frontend. After providing the parameters required by the report a Mobiliser job is created, which will trigger the creation of the report at the scheduled times.

6.4.1 Report Services

The report service provides a list of web service methods to use existing report templates to create and retrieve specific reports. This chapter lists the provided service methods from a business logic view. Please see the Reporting sections of the Mobiliser service reference for the detailed request/response parameter documentation.

GetAvailableReports List all the reports (templates) available in the system.

CreateReport Create a report based on the given template and report parameters. The generated report is returned immediately by this call (synchronous processing).

CreateAsyncReport Triggers the report generation. It returns a report key that is used later in the `GetGeneratedReport` method request to retrieve the report.

GetAvailableGeneratedReports Returns a list of generated reports by the report job for the caller, based on its report privileges.

GetGeneratedReport Returns a asynchronous generated report (either created by the report job or `CreateAsyncReport` service)

6.4.2 Report Job

The report job uses the standard Mobiliser job scheduler and service to register a report scheduler which is configurable through the database, starts report generation through the reporting service per configured schedules, and supports pluggable workers to deliver the generated reports.

The cron job handler name is exposed with:

```
jobKey=com.sybase365.mobiliser.util.report.watcher.ReportJob
```

The format for tasks data is JSON, e.g.

```
{"name":"TestReport1","locale":"en_US","format":"PDF", "reportParameters":[
  {"value":"v1","key":"p1","type":"java.lang.String"},
  {"value":1320702460403,"key":"p2","type":"java.util.Date"}
]}
```

where the "name" parameter must match the report name already provisioned in the system. Also note that the date parameter values are in milliseconds from epoch. Table 6.1 lists the JSON parameters, which are parsed by the job.

6.5 Report Store

The report store is used to store report templates as well as generated report instances. The live directory may hold any number of subdirectories. All reports in one sub-directory are logically grouped together and require the same user privilege to actually use these templates and generate reports out of them.

The live sub-directories holds report templates that are available through the report services (and hence also through the front-end). The monitored directories together with the required privilege for actually being able to

Name	Type	Description
name	java.lang.String	The Report Name
lastModifiedTime	java.lang.Long	(optional) Date of last modification.
locale	java.lang.String	(optional) The report Locale
format	java.lang.String	The report output format. Available formats are CSV, XLS, PDF
owner	java.lang.Long	The report owner / customer id. If not set the generated report is available globally
reportParameters	reportParameters[]	A list of report parameters (value: key: type) see example

Table 6.1: Common Report Parameters

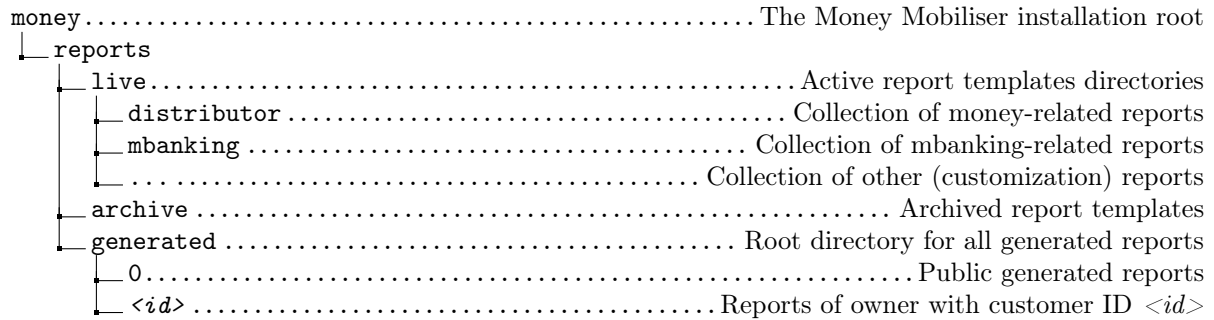


Figure 6.3: Directory tree for the report store, templates and archives

use these reports is configured through the ConfigAdmin file. Each directory can have an associated single UI privilege that decides the access rights to the execution of the report. The privileges are provided a comma-separated entries from the report watcher properties, e.g.:

```

#list of poll directories
pollDirectory=${mobiliser.home}/reports/live/distributor,
    ${mobiliser.home}/reports/live/mbanking
#poll directory privilege - matches privilege to poll directory
pollDirectoryPrivilege=MERCHANT_PORTAL_REPORTS,UI_CST_MBANKING_REPORTS

```

There should be a match of number of privilege entries to the number of directory entries. In case the framework cannot process reports or a report should be disabled manually, they are moved into the archive directory. Asynchronous and batch reports, which are stored until the user downloads it, are held on the server. The reports are stored in directories which, by convention, have the customerId of the customer requesting the report as folder name.

Chapter 7

Events

The Mobiliser components all have individual requirements to respond to specific states that may occur during the life cycle of requests or transactions. Together, they can be viewed as states that require actions that are normally processed out-of-band of the request or transaction itself, but nonetheless have to be processed with a potentially important outcome. Typically, these processes have been serviced by an *handling mechanism* that allows an actionable response to a known situation.

The event system basic set of requirements are;

- Loosely coupled system - detach producer, consumer, processing, store and schedule.
- Integration with AIMS - Use the AIMS/OSGi services for bringing together the events system.
- Performance - Stable and distributable.
- Improved Data Handling - Allow for more event data.
- More Event Types - Support different aspects of events .

The internal event handling system in Mobiliser introduce the AIMS/OSGi service architecture to event handling plus a redesign of how events are raised, event data management and the event handling process APIs. The Mobiliser event handling system is also intended to be used throughout the Mobiliser AIMS/OSGi architecture based products; for example, it is to be used in Brand Mobiliser.

The event handling system as described by this document only supports the concepts of a simple event processing. The concepts and use of advanced event driven architectures, such as event stream processing and complex event processing are not out of scope of further extensions to this design in future. However, both event stream and complex event processing require an effective simple event system to interact with in the first place.

The Mobiliser event driven system perfectly complements the service oriented architecture of both AIMS/OSGi and the Money Mobiliser system architecture: Internally to the event handling system, core functions such as event storage and scheduling are provided by services. Within the events themselves, services can be initiated by actions triggered from events, including cascading of actions and invocation of Mobiliser core services.

The new Events system described in this document also encompasses a task scheduling capability that builds on the events system in order to action the processing of the task. A Task is fundamentally different from an Event because it's task configuration and action is self-described in the task handler itself, removing any need for an Event to be defined for the Task configuration itself. Aspects of Task processing are described in a separate section.

7.1 System Overview

Figure 7.1 shows how the event **Generator Applications** are loosely coupled from the **Event Handler Applications** by using different access points into the **Event Processing** (the event-core bundle). The **event-core**

maintains the components:

- Class model for event generation; through the **Event** API marked in the diagram above.
- Internal logic for event processing; maintained within the **event-core** bundle.
- Service interfaces for event processing; through the `StoreProvider` service and `SchedulerProvider` service marked in the diagram above.
- Class model for event handling; through the `Handler` API marked in the diagram above.

External from the event-core the `SchedulerProvider` service also exposes an alternative interface to allow the creation and configuration of scheduled tasks.

A `Task` is defined as an action scheduled for execution at known repeated intervals, as defined and processed by the **Task Handler**.

A `Task` is not directly related to an event because a `Task` is never stored in the event system nor requires the regeneration of historical events for a task handler. However, the processing of the task action is initiated and controlled through the event system itself.

The following sections describe the processing flows involved in the components.

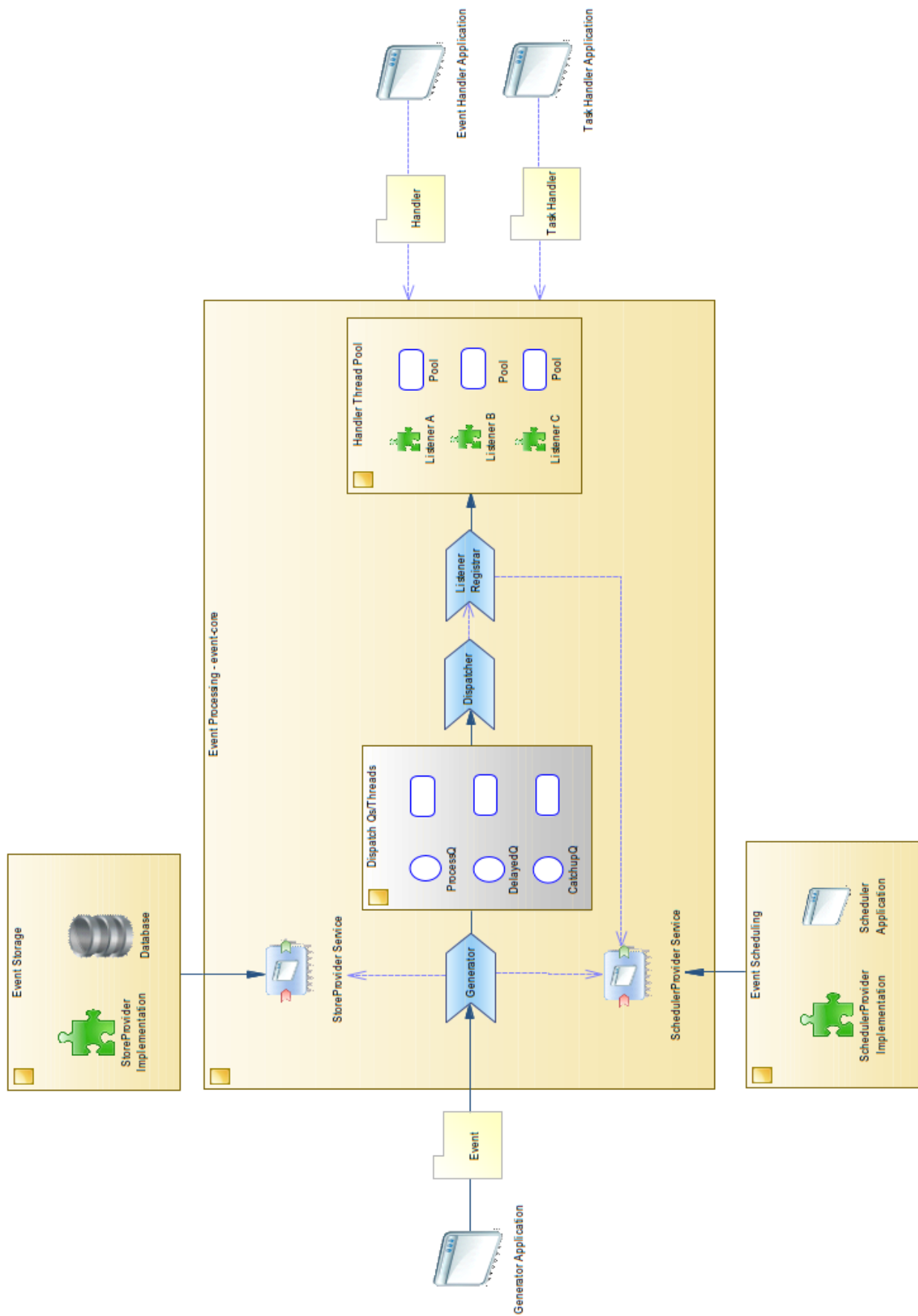


Figure 7.1: Overall System Overview

7.2 Event Generation

Events are generated by producer applications and services using the model as shown in overview below.

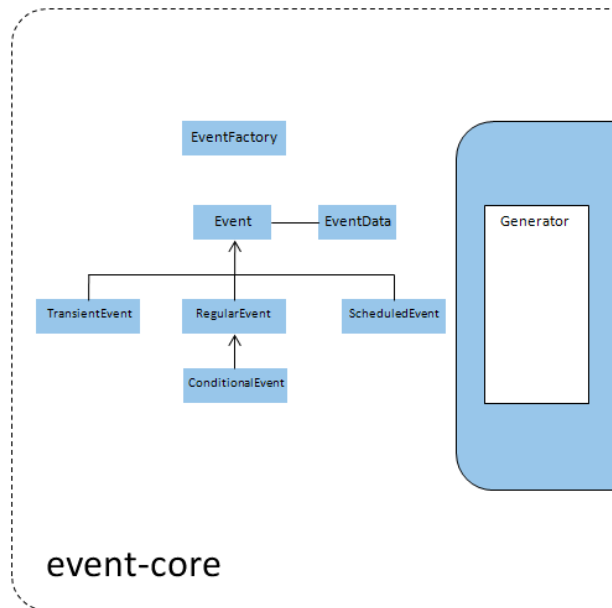


Figure 7.2: Generation Class Model Overview

The following types of event are known to the event system;

- **RegularEvent** – a normal event, without any special needs in the event system.
- **ConditionalEvent** – an event that will only be generated if a specified condition associated with the event data is met.
- **ScheduledEvent** – an event that will occur at a fixed point in the future and optionally will repeat.
- **TransientEvent** – an event that is transient in nature; will occur only in memory and never stored.

All events have an **event name** and an **event body**, identified by its associated `EventData`. The `EventData` manages a set of property values identified by a key.

A scheduled event also has some other data associated with it to identify the scheduled date/time and the repeating interval, as necessary.

All event types other than a scheduled event also have the option of specifying a short time period that allows the delay of processing of the event for that period of time. If the `EventDelay` is not specified for an event, then the event will be attempted to be processed immediately.

An event producer application has the following options for generating an event.

- Create an anonymous sub-class of the specific event class using the `EventFactory`.
- Sub-class the specific event type implementing the required abstract methods.

7.2.1 Disable Event Generation

The event generator can be configured to disable the event generation for a list of event types. If a events name is part of the comma separated list specified by the property `disabled.events` the event is not generated and persisted in the event table. The list of disabled events is read during bundle start and **not updated** dynamically.

7.3 Event Processing

This section describes the internal event system processing that occurs in the event system between event generation and event handling. This section is mainly for information only, as the developers need not understand all the internal details of event processing in order to generate events or develop event handlers.

7.3.1 Event Generation Processing

An event is created by using either:

- The EventGenerator OSGi service instance method `EventGenerator.create(Event)`.
- The abstract super-class `Event.create()` method.
- The static method `EventFactory.create(Event)`.

In each case, this passes the event into a **Generator** singleton instance, which first validates the event, as shown in the diagram below.

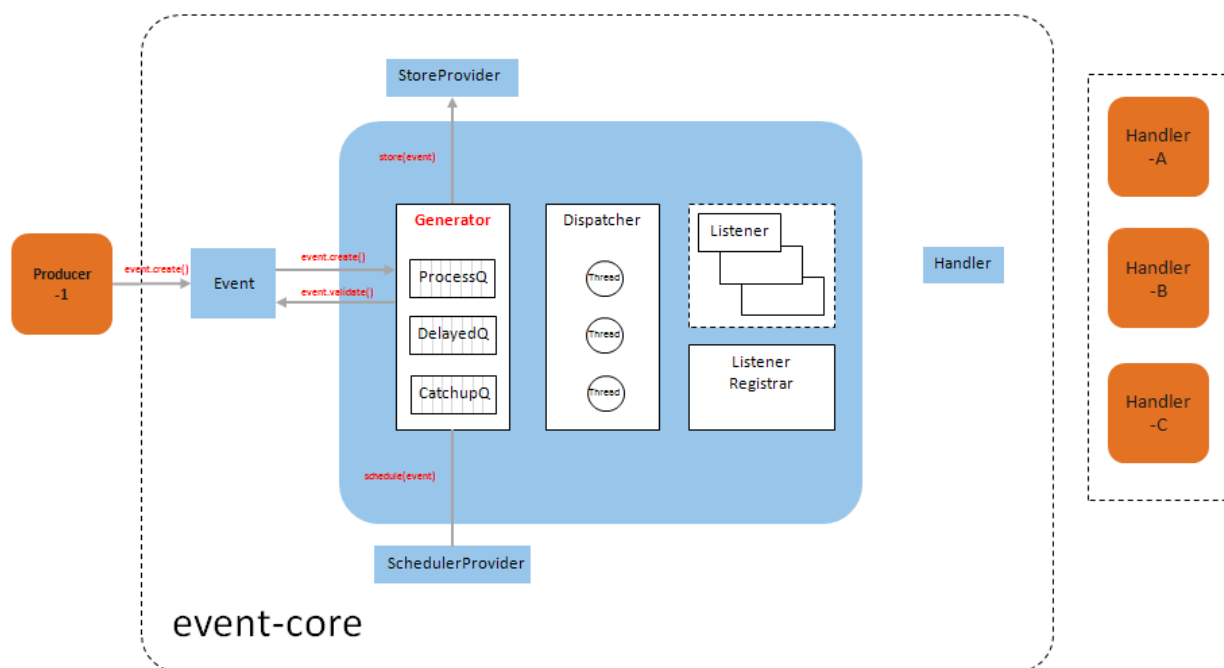


Figure 7.3: Event Generation Processing

Validation depends on the class of event and any condition criteria associated with the event.

If validated, the event is then stored – it is up to the `StoreProvider` service to map event data to the database – and a unique identifier is generated (likely the PK of the database table entry).

If the event is a `ScheduledEvent` the event is passed to the `SchedulerProvider` for future triggering and no more event dispatching is needed.

If the event is not a `ScheduledEvent`, the event passes into the dispatch processing, as described below.

7.4 Event Dispatch Processing

For all non-ScheduledEvent's, the event is triggered and placed onto an in-memory queue; either the ProcessQ or the DelayedQ;

- The ProcessQ is a LinkedList queue – an unbounded FIFO queue.
- The DelayedQ is a java.util.concurrent.DelayedQueue – an unbounded ordered queue, ordering of elements provided by the Delayed interface. Used only when there is a EventDelay associated with the event.

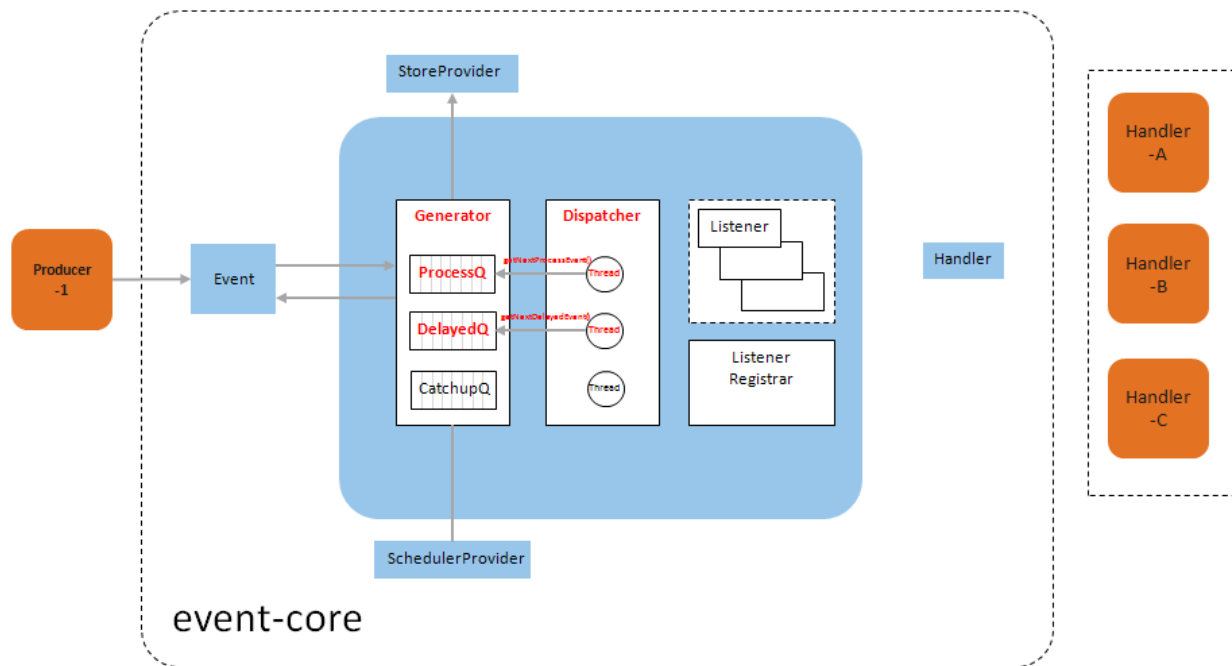


Figure 7.4: Event Dispatch Processing #1

Note: The CatchupQ is also a LinkedList queue – an unbounded FIFO queue, but not directly used during normal event creation and dispatching. It is used by the event regeneration process.

The **Dispatcher** manages three dispatch threads directly associated with each individual queue, as show in the diagram above.

Note: The queues used are not sized and are unbounded. However, in configuration there is the option to specify a virtual capacity. When a virtual capacity is reached warnings are output to the log file and the virtual capacity is increased. This means processing will continue (and associated memory usage will grow), but notifications should be given and heeded.

When the **Dispatcher's ProcessQ** polling **thread** receives an event it will first look at the re-queue settings for the event.

If the event has never been re-queued, then a lookup is done via the **ListenerRegistrar** of all installed and recognized event handler **Listeners** for this event type.

If there are no **Listeners** known then the event will be ignored. However, It may be processed sometime in the future, due to catchup processing of an event handler, but this is dependent on the event handler as described in a later section.

If there are **Listeners** for the event, then the **Dispatcher** will invoke the process method on each **Listener** to start event handling processing. (using the thread pooling strategy associated with the handler, as described in a later section).

If the event has been re-queued due to catchup processing, then a specific event handler **Listener** will be noted on the event header. Therefore, only a lookup for the named event Listener is required.

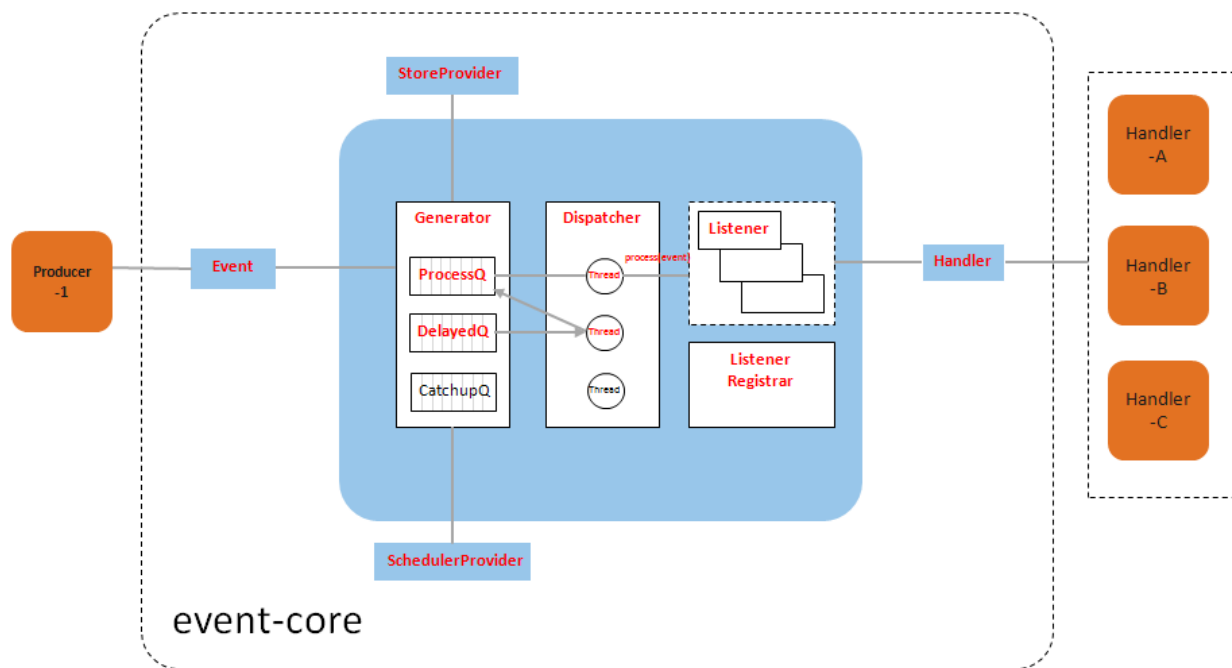


Figure 7.5: Event Dispatch Processing #2

When the **Dispatcher's DelayedQ** polling **thread** receives an event it will requeue the event onto the **ProcessQ** for immediate processing, which then follows the same steps as described above for non-delayed events.

7.4.1 Handler/Event Process Lock

Before starting the processing, the event handler is allocated to this event for processing by assigning a Event processing lock to the event and handler combination.

The implementation of the processing lock is dependent on the StoreProvider service, but will likely involve the entry of a database table row. If there already exists a database table row for this event and handler, then it may be assumed the event is already being or has been processed by an event handler, and so the process lock is not acquired. If there doesn't already exist a database table row, then one can be inserted and then the processing lock can be assumed to have acquired successfully.

After the event handler is complete for event, the handler/event status is then updated to indicate the success or failure of the handlers processing. The existence of a handler/event database table entry is important for catchup processing, since it identifies events that have been processed by a handler.

7.4.2 Processing of Scheduled Events

When scheduled events reached their scheduled date/time or repeat date/time, they are triggered for processing by the SchedulerProvider.

The processing of a scheduled event trigger involves generation of a **RegularEvent** with the same event **name** and event **data** as the original ScheduledEvent. For this instance of a RegularEvent, processing continues as described above.

7.5 Event Handling

For an event to be processed by a handler, there has to be;

- An event handler registered for the event name.
- An available thread from the process pool as determined by the event handler.

Note: There may be zero, one or many event handlers registered per event name. If there are many, they are not processed in any defined order.

Note: A single event handler instance is associated with a single event name only. If the same event handler needs to process different event names, then multiple instances will need to be registered, with different handler names.

The following sections describe the two different aspects of handler registration and handler pooling.

7.5.1 Event Handler Registration

Event handlers are registered to the event system through the OSGi service registry.

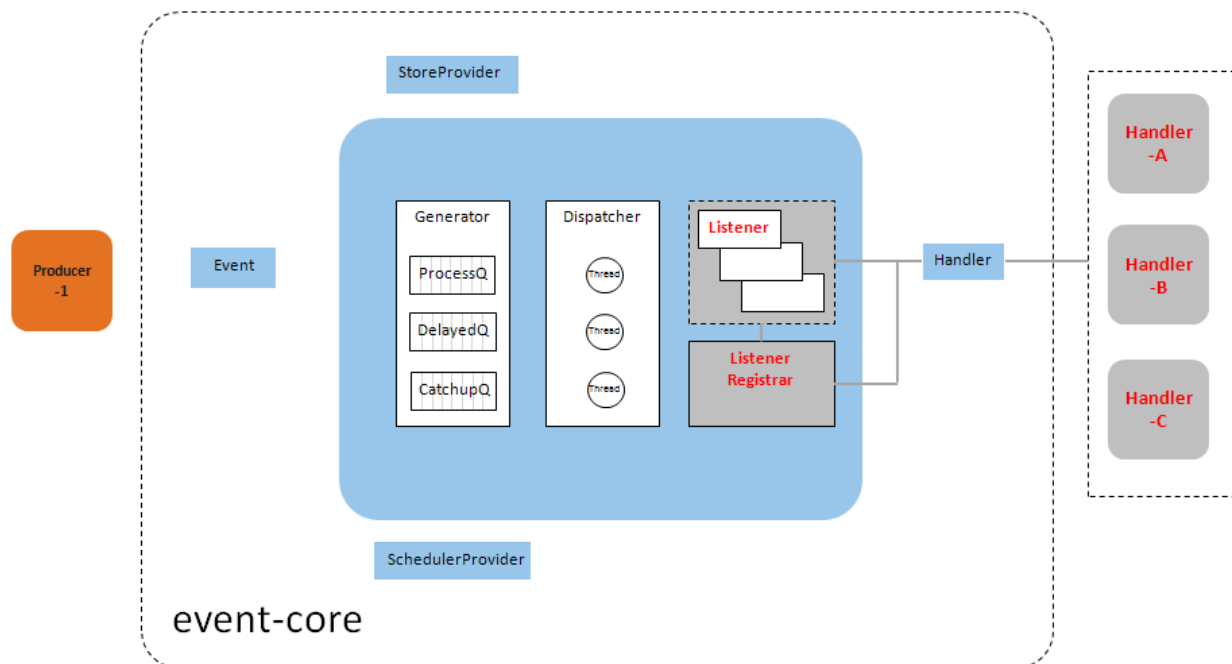


Figure 7.6: Event Handler Registration

All event handlers implement (indirectly) the **Handler** interface and register through Spring Beans and Spring OSGi service declarations into the OSGi environment.

The `ListenerRegistrar` receives the `eventServiceRegistered` and `eventServiceUnregistered` events for dynamically installed handlers (or stopped and started event handler bundles), or on startup of the event system it receives a list of known handlers already installed.

7.5.2 Event Handler Polling

All event handlers must directly extend the two pooling-type handler classes;

- SerialHandler – for event handlers that will run only one thread of execution.
- ParallelHandler – for event handlers that allow a pool of threads for execution.

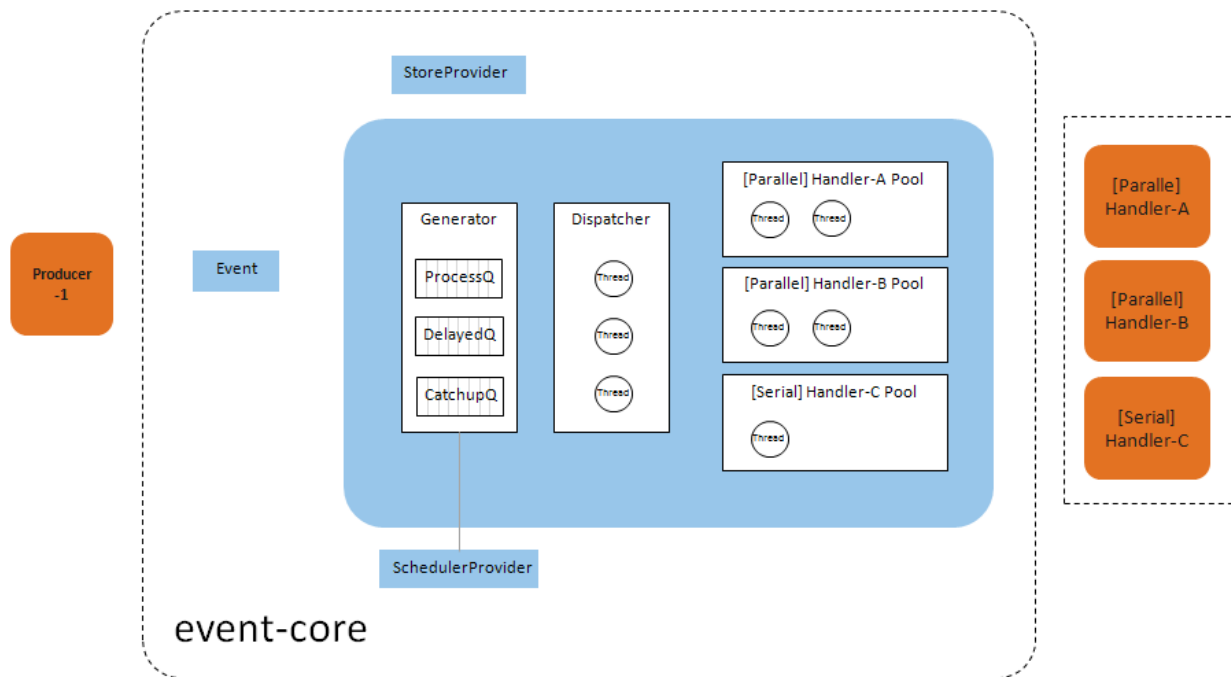


Figure 7.7: Event Handler Pooling

Note: Pools are associated with named event handlers, so each uniquely named handler has it's own pool.

Note: A SerialHandler simply uses a pool size of one thread maximum.

If the Dispatcher cannot get an available thread from the handler's thread pool because all the threads in the pool are actively processing other events handled by the same handler, then the event is placed onto the DelayedQ with a small delay of 500ms. The event handler that was attempting to process the event is marked into the re-queued event. Therefore, another attempt to process this event will be made in 500ms.

7.6 Event Catchup

Event handlers decide through the Handler API if they require processing of events that were generated at a point in time when the handler was not registered. This can happen if the event handler is newly registered or it's bundle was not active at the point in time the event was generated.

If there is no active handler at time of creation of an event, the event is just stored.

Handlers provide an expiry period - through `Handler.getExpiryPeriod()` - after which their handled events will be deemed to be expired.

Note: An expiry period of "0" (zero) means that events are only handled if active and catchup is not required.

The **Handler** catchup process starts when a Handler is installed – the **ListenerRegistrar** (through the **Generator**) will regenerate events that haven't yet been handled by the **Handler**, but are still within it's expiry period.

It does this `Generator.regenerate()` call once, then if the regeneration hit a configurable regeneration batch limit, a separate **Catchup Controller** thread will be started to asynchronously check when that batch has been processing to generate a new batch by running another `Generator.regenerate()` call, if necessary.

The **Listener** for the **Handler** is placed in **CATCHUP** status and regenerated events are dispatched via the **CatchupQ** to the **Handler**.

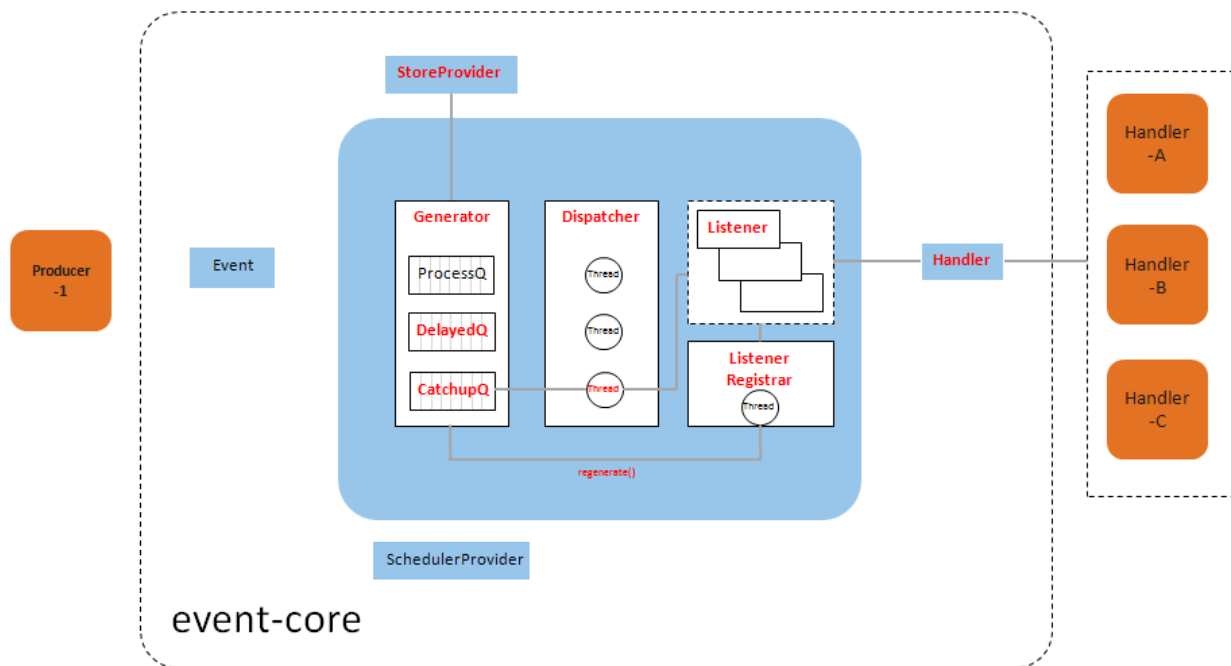


Figure 7.8: Event Handler Catchup

Regenerated events are tagged in the event header for processing by a specific named handler. Normally, when events are first created they may be consumed by a number of handlers. However, the regeneration process is specific to a handler/event.

Regenerated events are recreated in batches – determined by a batch size in the generator passed to the **StoreProvider**. This will ensure large amounts of regenerated events do not swamp the db processing involved in the **StoreProvider** or the size of the **CatchupQ**.

During catchup processing new regular events may be created, regenerated delayed events may be ready to process, or scheduled events may trigger. These events are not part of the catchup process, so they get assigned to the **processQ** and they will be processed alongside the catchup events.

7.7 Task Configuration

The creation of a **Task** is different from that of an event;

- Tasks have a closely coupled **1-to-1** relationship between it's creation and it's handler.
- Events have a loosely coupled **1-to-0..N** relationship between event producers and consumers.

Therefore, a **Task** is directly associated with it's own creation and handling. This means there is a slightly different process for task configuration and the installation of task handlers into the system.

7.7.1 Task Handler Registration

Task handlers are registered to the event system through the OSGi service registry.

All task handlers implement (indirectly) the **TaskHandler** interface and register through Spring Beans and Spring OSGi service declarations into the OSGi environment.

The **ListenerRegistrar** receives the **taskServiceRegistered** and **taskServiceUnregistered** events for dynamically installed task handlers (or stopped and started task handler bundles), or on startup of the event system it

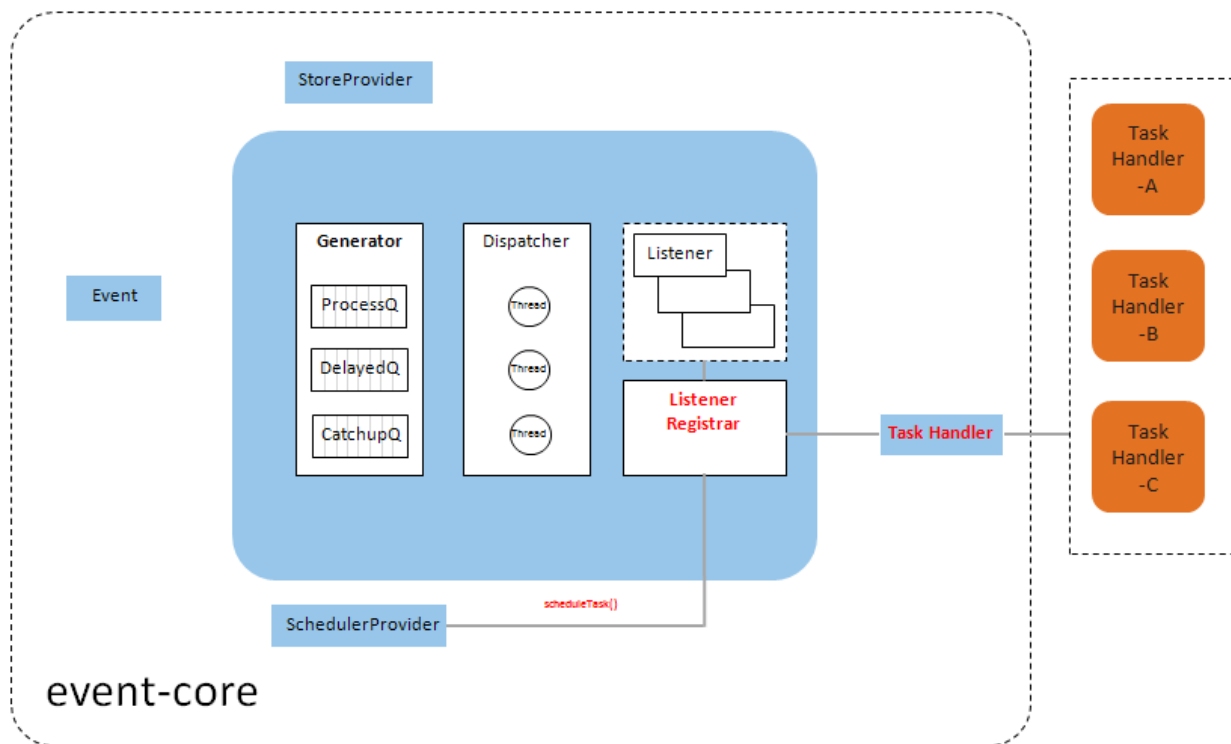


Figure 7.9: Task Handler Registration

receives a list of known task handlers already installed.

At the point of registration of the **TaskHandler** the **ListenerRegistrar** uses the **SchedulerProvider** to schedule a repeated task that will trigger based on the configuration available from the **TaskHandler**.

7.7.2 Task Configuration

The Mobiliser tasks system allows the configuration of both;

- A repeatable scheduled task run – this will trigger based on a schedule provided.

Both task types are created through the **SchedulerProvider** OSGi service instance.

```

1 ...
2 public interface SchedulerProvider {
3 ...
4 /**
5  * Create a repeated scheduled task
6  */
7 public void scheduleTask(String name, String cronExpr, Map<String,String> data,
8                          String data, String timezone);
9 /**
10 * Cancel an existing one-off or repeated task
11 */
12 public void cancelTask(String name);
13 }

```

Listing 7.1: com.sybase.mobiliser.framework.event-scheduler.SchedulerProvider

Each of the items of task configuration required for the scheduler are provided by the task handler interface as described in the “Mobiliser Framework Development Guide”.

Note: Repeatable scheduled tasks are configured using a Cron Expression. These are typically used and understood by system administrators in Unix environments for configuring repeated tasks at the system level using the Cron system scheduler. In the `SchedulerProvider` interface, the Cron expression is used because it is a generally understood format and not because it uses an underlying Cron system scheduler itself.

7.7.3 Task Processing

When a scheduled task action triggers it does the following;

1. The **SchedulerProvider** service generates a event system `TransientEvent` (described in section 3.6 Transient Events). This type of event is used when there is no requirement for storage or regeneration of the event. In this case this event is the link between the scheduler provider trigger and the task handler.
2. The Transient Event created is given the name of the task name that caused the trigger to fire.
3. The Transient Event is initialised with the task data that was associated with the task.
4. The Transient Event is passed into the event system for processing.
5. The event system processes this event in the same way as other events using the same handler registration and thread pooling mechanisms.
6. The task handler (if any) installed for this task name is actioned by the event system process.

Bibliography